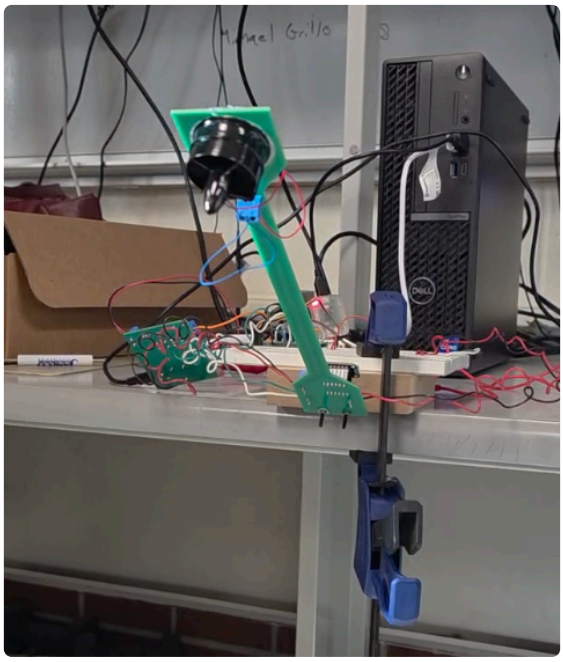
ECE 5730 · DIGITAL SYSTEMS DESIGN USING MICROCONTROLLERS

Lab 3 Report

PID Control of a 1D Helicopter

Amber Li | Ivan Huang | Zarif Karim

Thursday lab section · 11.08.2026



The arm holding +30° above horizontal during the demo.

Contents

1 Introduction

2 The Math and Physics

2.1 Measuring the angle · 2.2 The complementary filter · 2.3 How the arm moves · 2.4 PID control · 2.5 PWM and the motor circuit · 2.6 Fixed point and I2C

3 How We Built and Tested It

3.1 Build order and bugs · 3.2 Motor driver · 3.3 PWM and IMU setup · 3.4 The control loop · 3.5 Overcurrent protection and windup · 3.6 Demo button · 3.7 VGA and serial · 3.8 Wiring

4 Documentation

5 Results

5.1 Final gains · 5.2 Demo sequence · 5.3 Settling times · 5.4 Hover duty vs. angle · 5.5 Stable range · 5.6 Earlier tuning runs

6 Conclusion

7 Answers to the Lab Report Questions

8 Work Distribution and AI Use

References

Appendix A Commented Code Listing

Appendix B AI Prompt Log

1 Introduction

Our "helicopter" is a lever arm on a hinge with a small ducted fan at the end. The goal is to make the arm go to an angle we pick and stay there.

Here's how it works. An MPU-6050 sensor on the arm measures acceleration and spin rate. The Pico 2 combines the two readings to figure out the arm's angle. A PID controller then decides how hard to run the fan, 1000 times a second, and sends that out as a PWM signal through an isolated motor driver.

We can change the target angle and the three PID gains from a serial terminal while it runs. A VGA screen plots the angle, the target and the motor command live. Pressing and releasing a button runs the demo: the arm starts hanging straight down, then goes to 0°, +30° at 5 s, −30° at 10 s and back to 0° at 15 s.

With our final gains ($K_p = 80$, $K_i = 0.2$, $K_d = 12$), the arm hit every demo angle, settled in 1–3.6 s, and stayed stable anywhere from −90° up to about +50°.

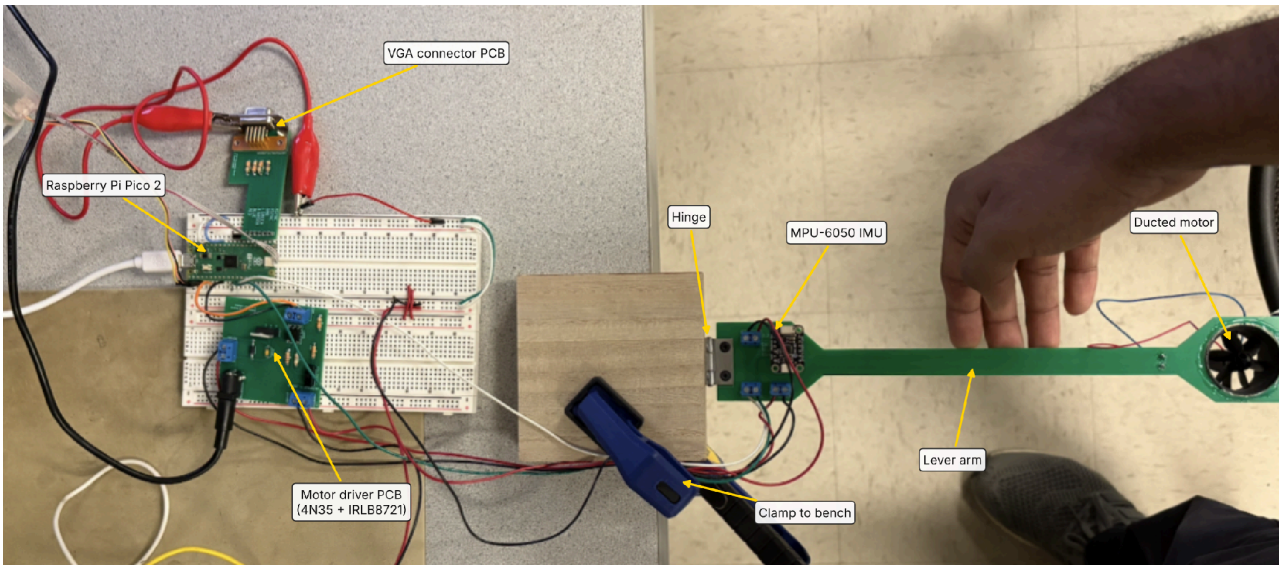


Figure 1. The setup from above: Pico 2 and VGA adapter on the breadboard, the motor-driver board, the hinge clamped to the bench, the IMU next to the hinge, and the fan at the end of the arm.

> **Angles in this report:** 0° means the arm is level, −90° means it's hanging straight down, and positive means above level. (The course's analytical notes measure θ from hanging down instead, so when we borrow their equations, $\theta = \varphi + 90°$, where φ is our angle.)

2 The Math and Physics

2.1 Measuring the angle

We have two sensors, and each one is good at a different thing.

The accelerometer feels gravity. When the arm is still, the only thing it measures is gravity, so the direction of that reading tells us which way is down, and so how tilted the arm is (Figure 2a). The arm rotates about the sensor's y axis, so gravity shifts between the x and z axes. Since the arm swings through well over 90°, we can't use a small-angle shortcut; we use the full arctangent:

$$\theta_{acc} = \frac{180}{\pi}\,\mathrm{atan2}(-\bar{a}_x,\ \bar{a}_z)$$

The good part: gravity never goes away, so this angle never drifts. The bad part: the sensor can't tell gravity apart from any other push. Once the fan is running, vibration and the arm's own motion make this reading really noisy (Figure 19).

The gyroscope measures how fast the arm is rotating (°/s). Add up that rate over time and you get an angle that's smooth and ignores vibration. The catch is that every gyro has a small built-in error b, and adding it up over time makes the angle slowly drift away (after t seconds it's off by about b·t). Each step we just add rate × time:

$$\theta_{gyro,\,n} = \theta_{gyro,\,n-1} + \omega_{y,\,n}\,\Delta t \qquad (\Delta t = 1\ \mathrm{ms})$$

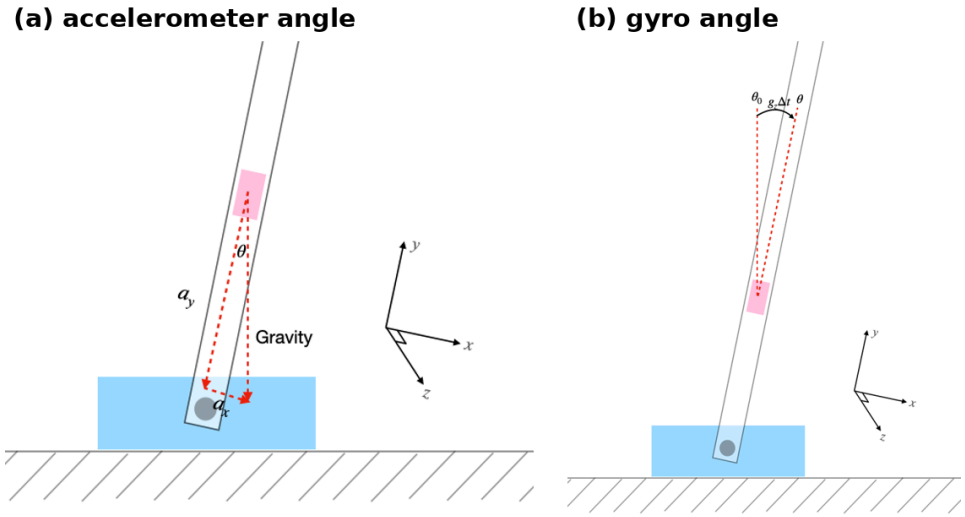


Figure 2. (a) The accelerometer angle comes from the direction of gravity. (b) The gyro angle adds up rate × time each step. From the course notes, which call the axes a_y and g_z; on our board they're a_z and gyro y.

2.2 The complementary filter

So the accelerometer is right in the long run but jumpy, and the gyro is smooth but drifts. The complementary filter uses both. Every 1 ms, it takes the last angle, adds the gyro's change, and then nudges the result 0.1% of the way toward the accelerometer's angle (Figure 3):

$$\hat{\theta}_n = 0.999\left(\hat{\theta}_{n-1} + \omega_{y,n}\,\Delta t\right) + 0.001\,\theta_{acc,n}$$

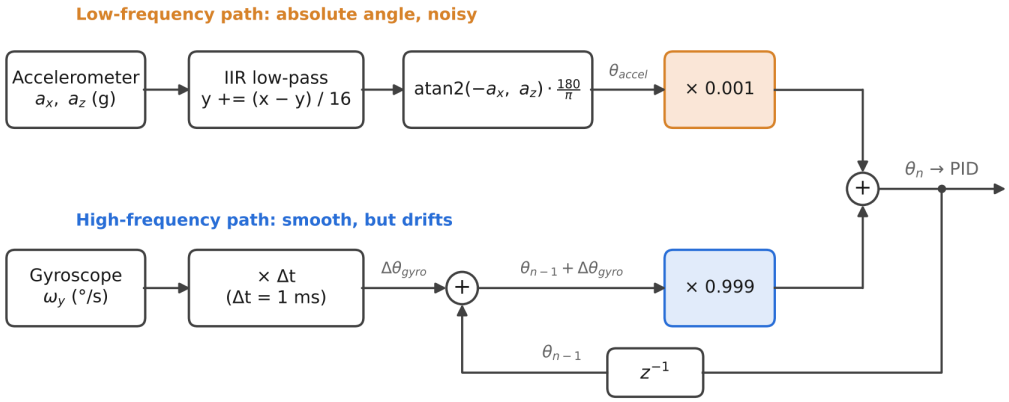


Figure 3. Our complementary filter, as written in `on_pwm_wrap()`.

Why "complementary"? The two weights add up to 1. For slow changes (slower than about 1 second, the filter's time constant τ) we trust the accelerometer, and for fast changes we trust the gyro. The real angle gets through untouched; only each sensor's own noise gets filtered out.

- **Gyro drift turns into a small fixed offset** (about τ·b) instead of growing forever. A gyro error of 1°/s would leave us about 1° off.
- **Our weights.** The notes suggest 0.99/0.01. We used 0.999/0.001, which leans harder on the gyro and keeps more fan vibration out, at the cost of a bigger drift offset. With the fan running, the accelerometer readout bounced 2–5° away from the filtered angle while the filtered angle stayed smooth (compare the two readouts in Section 5.2).

Smoothing before `atan2`. Noise doesn't average out nicely after a nonlinear function like `atan2`; it shifts the result. So we first smooth the raw a_x and a_z with a simple digital filter that works like an RC circuit (in the code it's a right-shift by 4):

$$y_n = y_{n-1} + \frac{x_n - y_{n-1}}{16} \qquad (\text{cutoff} \approx 10\ \text{Hz})$$

That adds about 16 ms of delay, but only to the accelerometer, which only has 0.1% say in the answer. The gyro path has no delay. That's the whole reason to use a gyro: if we tried to just heavily smooth the accelerometer instead, the delay would land inside the control loop, and delay is what makes feedback loops unstable.

2.3 How the arm moves

The motor speeds up and slows down much faster than the arm swings, so for our purposes the fan speed just follows the voltage. The arm is the slow part, so the arm is what we have to control.

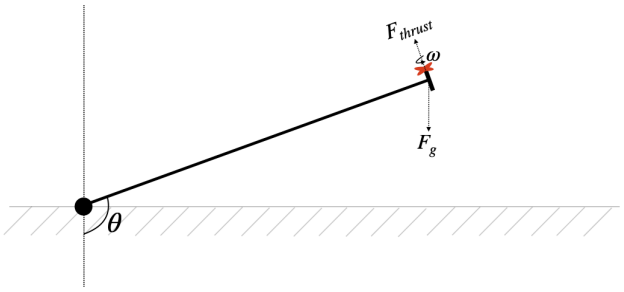


Figure 4. Thrust and gravity on the arm, with θ measured from hanging down. From the course PID notes.

The arm. Three torques act on it: thrust from the fan (about $K_T\omega^2$), gravity pulling the fan's mass down, and friction in the hinge. Thrust minus gravity minus friction equals how fast the arm speeds up:

$$K_T \omega^2 L - mgL \sin\theta - K_f \dot{\theta} = mL^2 \ddot{\theta}$$

For small wiggles around one angle, the arm acts like a mass on a spring, where gravity is the spring. That explains a few things we saw in lab:

- **Hanging down,** the arm is just a pendulum. It swings back and forth on its own and barely damps out.
- **The spring flips at horizontal.** Below level, gravity pulls the arm back toward hanging. Above level, gravity pushes it further away. So above level, the controller is the only thing keeping the arm up.
- **The arm responds differently at different angles.** How much one extra duty count moves the arm depends on how fast the fan is already spinning, so it depends on where the arm is. One fixed set of gains can't be perfect everywhere, which is part of why the stable range has limits (Section 5.5).

Thrust needed to hover. When the arm is sitting still, thrust exactly cancels gravity (φ is our angle, 0° = level):

$$K_T \omega_0^2 = mg \cos\varphi_0$$

For a simple arm with all the mass on its centre line, that means the most thrust at level and the same thrust at $+30°$ and $-30°$. But on our rig the fan hangs *below* the arm (see the photo in Figure 14), so the centre of mass sits a bit below the arm's line, a distance d. Then gravity's torque becomes

$$\tau_g(\varphi) = mg\,(r\cos\varphi + d\sin\varphi)$$

and the arm needs *more* thrust above level than below. Our measurements in Section 5.4 show exactly that.

2.4 PID control

A lookup table from angle to duty wouldn't work: the thrust needed changes with the supply, motor temperature and so on, and a table can't fix a bump. So we use feedback (Figure 5): measure the angle, compare it to the target, and turn the error into a motor command.

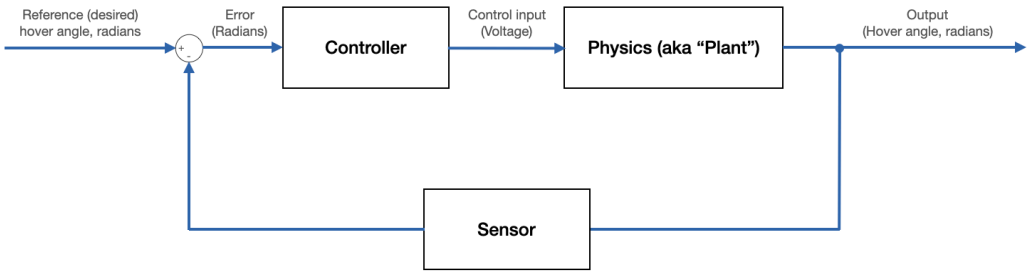


Figure 5. Closed-loop control. From the course PID notes.

$$u(t) = K_p\, e(t) + K_i \int_0^t e\, dt' + K_d \frac{de}{dt}, \qquad e = \theta_{des} - \hat{\theta}$$

- **P (proportional)** pushes harder the further the arm is from the target (Figure 6a). On its own it always leaves the arm a bit low: P only outputs thrust when there's error, and holding the arm up needs thrust. Too much P and the arm oscillates.
- **I (integral)** adds up the error over time, the area under the curve (Figure 6b). It keeps growing until the error is gone, so it fixes P's "a bit low" problem. Once the arm is settled, P and D are basically zero and the I term is the thing holding the arm up. Too much I makes the arm overshoot and wobble slowly.
- **D (derivative)** looks at how fast the error is changing. It brakes fast motion, which calms down overshoot and oscillation.

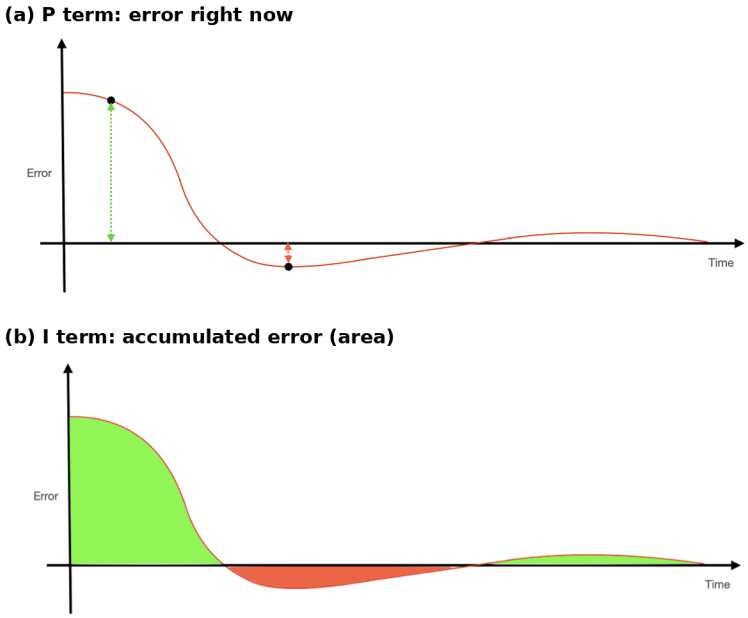


Figure 6. (a) P acts on the error right now. (b) I acts on the total area under the error curve. From the course PID notes.

Getting D from the gyro. The target only changes in jumps, so between jumps the rate of change of the error is just minus the arm's spin rate: $de/dt = -\omega_y$. So we plug the gyro reading straight into D, as suggested in lecture. Taking the difference between angle samples would be way noisier: at 1 kHz the angle barely changes from one sample to the next, so you'd mostly be measuring noise. A bonus is that D doesn't spike when the target jumps (no "derivative kick"), because the gyro doesn't know about the target.

What the code actually does every 1 ms:

$$I_n = \text{clamp}(I_{n-1} + K_i\, e_n,\ 0,\ I_{max}), \qquad u_n = \text{clamp}(K_p\, e_n - K_d\, \omega_{y,n} + I_n,\ \ 0,\ \ u_{lim,n})$$

u_{lim} is a ramp limit we explain in Section 3.5. The I term stores $K_i \cdot e$ added up (already in duty counts), so changing K_i on the fly never makes the output jump. In real units, our final gains are:

$$K_p = 80\ \frac{\text{counts}}{\text{deg}}, \qquad K_d = 12\ \frac{\text{counts}}{\text{deg/s}}, \qquad K_i = 0.2\ \frac{\text{counts}}{\text{deg}\cdot\text{ms}} = 200\ \frac{\text{counts}}{\text{deg}\cdot\text{s}}$$

What each gain does to stability (from the Bode plots in the analytical notes): more K_p makes the arm faster and closer to the target, but at some point it oscillates forever. We saw that at $K_p = 120$ (Figure 18). K_d adds back stability margin, which lets K_p go higher. K_i gets rid of the leftover error but costs some stability, so it stays small.

Common PID problems and how we handled them. The course notes warn about a few things that go wrong in practice. Table 1 lists them with our fix for each, and Figure 7 shows the worst one.

Problem	Our fix
Integrator windup: I keeps growing while the arm is held or the motor is maxed out, so the arm overshoots badly when it's let go	Only let I grow if the output isn't already at its limit. I can't go below 0 or above `i_max`, and it's reset whenever the loop is off (button held, `c 0`, manual duty) or K_i is set to 0
Taking a derivative amplifies noise	D uses the gyro rate directly
At 1 kHz the angle barely changes between samples	Same: the gyro measures rate directly
D spikes when the target jumps	D uses the gyro, which doesn't see the target
Fan vibration makes the accelerometer noisy	Smooth it before `atan2`, then give it only 0.1% weight
The fan can only push up, so a negative command means nothing	Output kept between 0 and 6000; I kept ≥ 0
Big current surge when the command jumps from 0 to full	Output can rise at most 20 counts per ms (Section 3.5)

Table 1. Practical PID problems and our fixes.

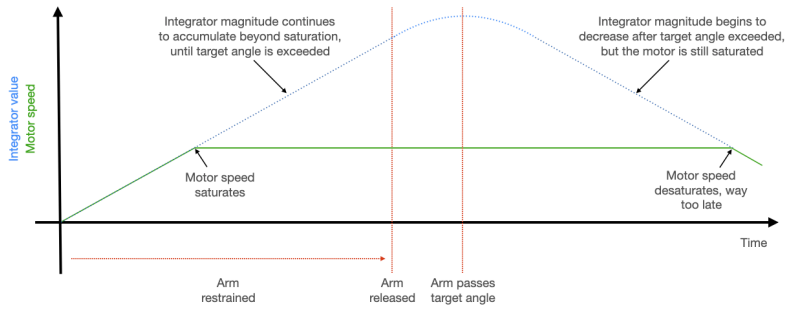


Figure 7. Integrator windup: I keeps growing after the motor is already maxed out, so the motor stays maxed long after the arm passes the target. From the course PID notes.

2.5 PWM and the motor circuit

PWM means switching the motor fully on and off very fast and changing what fraction of the time it's on. GPIO 4 is channel A of PWM slice 2. Its counter counts to 6000 and starts over:

$$f_{PWM} = \frac{f_{sys}}{\text{CLKDIV}\,(\text{WRAP}+1)} = \frac{150\ \text{MHz}}{25 \times 6001} \approx 999.8\ \text{Hz}$$

The motor is way too slow to follow 1 kHz switching, so it just responds to the average voltage. A duty level of 3280 out of 6001 is like giving the motor 2.7 V:

$$\bar{V}_{motor} = D\,V_s = \frac{\text{level}}{6001}\,V_s, \qquad \frac{3280}{6001} \times 5\ \text{V} \approx 2.7\ \text{V}\ (\text{hover at } 0^\circ)$$

Why only 1 kHz? The 4N35 optoisolator is slow (it takes tens of microseconds to switch), so a much faster PWM would never fully turn the MOSFET on or off. We also use the same counter wrap to trigger our control loop, so the controller runs at exactly the PWM rate without needing a separate timer.

Why the motor needs protection. A motor coil is an inductor, and inductors hate having their current cut off suddenly. The notes' example (1 mH, 1 A, switched off in 10 ns) gives a huge voltage spike:

$$V = L\frac{dI}{dt} = 10^{-3}\ \text{H} \cdot \frac{1\ \text{A}}{10^{-8}\ \text{s}} = 10^{5}\ \text{V}$$

The 1N4001 diode across the motor gives that current somewhere safe to go. The 0.1 μF ceramic capacitor soaks up high-frequency noise from the motor brushes. And the 4N35 keeps the Pico completely separate from the motor side: they share no ground, and only light crosses between them (Section 3.2).

2.6 Fixed point and I2C

Fixed point. Floating-point math is slow on the Pico, so the ISR does almost everything (except `atan2`) with `fix15`: a 32-bit integer where the bottom 16 bits are the fraction (Figure 8). 1.0 is stored as 65,536.

$$x_{fix} = \text{round}(x \cdot 2^{16}), \qquad (a\,b)_{fix} = (a_{fix}\,b_{fix}) \gg 16$$

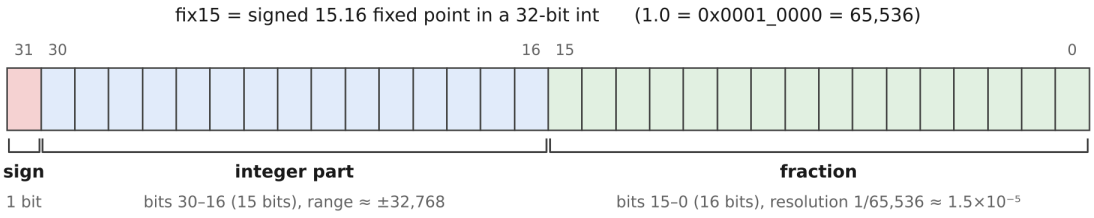


Figure 8. fix15 bit layout used by `mpu6050.h`.

To multiply, do the product in 64 bits and shift right by 16 (in the PID code we shift by 32, which gives a whole-number duty count directly). Constants get rounded (0.001 becomes 65), which makes the gyro step about 0.8% short, but the accelerometer slowly corrects for that. Two of our bugs came from mixing number formats (Section 3.1).

I2C is the two-wire bus the IMU talks on. Devices can only pull the wires low; resistors pull them back high. To read the IMU, the Pico sends the register address and then reads 6 bytes back. At 400 kHz:

$$t_{read} \approx \frac{(2+7)\ \text{bytes} \times 9\ \text{clocks}}{400\ \text{kHz}} \approx 0.2\ \text{ms}, \qquad 2\ \text{reads} \approx 0.4\ \text{ms of each } 1\ \text{ms}$$

So reading the sensor takes about 40% of every 1 ms loop. That's why the rest of the ISR is lean integer math and all the drawing happens on the other core. The "only pull low" design also explains our worst bug: if the Pico resets in the middle of a read, the IMU can get stuck holding the data line low, and every transfer after that hangs (Section 3.1).

3 How We Built and Tested It

3.1 Build order and bugs

We built it one piece at a time and checked each piece on the VGA screen before moving on. Week 1: motor driver board and plain PWM. Week 2: IMU, complementary filter, display and P control. Week 3: D, I, the demo button and overcurrent protection. Most of our debugging time went to start-up hangs and sensor axes, not the controller itself:

What we saw	Why	Fix
Fan at full speed even at duty 0 after merging the PWM code into the IMU code	The program never got to the PWM setup: the IMU reset hung on I2C, leaving GPIO 4 floating	Noticed VGA and serial were dead too; re-seated the IMU wiring
VGA blank and serial dead again, with the wiring fine	Stuck I2C bus: the IMU was holding the data line low after a reset mid-transfer. Reflashing doesn't fix it; unplugging does	At start-up, pulse the clock line up to 9 times until the IMU lets go (`main()`, lines 568–586)
Angles off by 2×	Leftover fixed-point macros in a different format	Deleted them
Angles off by a constant factor	`180/M_PI` chopped to 57	Use `oneeightyoverpi`
Angles read 80–300° and didn't follow the arm	We were using a_y, the axis the arm spins around (always near 0), plus a leftover +π	Use a_x, a_z and gyro y; remove +π so level = 0°
Filtered angle jumped the wrong way, then crept back	Gyro sign was backwards compared to the accelerometer (the symptom from lecture)	Add the gyro step instead of subtracting it
With P only, the arm sat about 10° low or bounced	P needs some error to make any thrust; more P oscillates	Add D (calms it), then I (removes the offset)

Table 2. Main problems we hit. Appendix B has the AI prompt log for these sessions.

Before closing the loop, we tested the filter by hand with the fan off: tilt the arm, watch both angle traces on the screen. That's how we caught the axis and sign problems.

3.2 Motor driver circuit

As required, the driver is soldered on the course PCB (Figure 9). The motor runs off its own 5 V supply through an IRLB8721 MOSFET with a 12 V gate drive. The Pico never touches the motor side: GPIO 4 lights the LED inside the 4N35 through 330 Ω, and the light switches the 12 V onto the MOSFET gate.

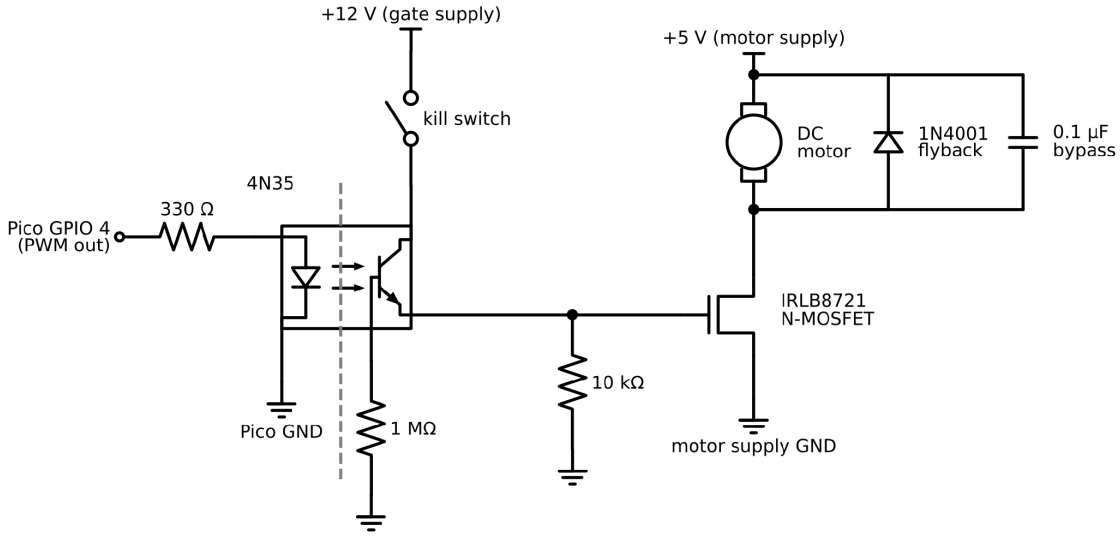


Figure 9. Motor driver schematic (as built on the course PCB). Only light crosses the 4N35; the two sides have separate supplies and grounds.

- **4N35:** GPIO 4 high → gate pulled up to 12 V → motor on. GPIO 4 low → the **10 kΩ** resistor pulls the gate to ground → motor off. That's the "extra 10 k" the course page asks about: without it the gate would float and the motor could turn on by itself. The **1 MΩ** resistor helps the 4N35 switch off faster.
- **1N4001 diode** across the motor catches the voltage spike from Section 2.5.
- **0.1 μF capacitor** across the motor soaks up brush noise.

We first tested the board with the course PWM demo and checked that the fan speed followed the duty cycle, then merged the PWM code into the IMU program (where the `n` command now sets a manual duty).

3.3 PWM and IMU setup

PWM. GPIO 4 and 5 are on PWM slice 2 (GPIO 5 is set to 0 and not used). The slice counts to 6000 with a clock divider of 25, which gives the 999.8 Hz from Section 2.5, and every wrap triggers `on_pwm_wrap()`. Max duty is 6000 (100%). We used a 4500 cap during early tuning and went back to 6000 once we added the ramp limit in Section 3.5.

IMU. The MPU-6050 is on I2C0 at 400 kHz (SDA GPIO 8, SCL GPIO 9), powered from the Pico's 3.3 V, and set up as in Table 3. The reads inside the ISR have 250 μs timeouts, so a glitch can cost one sample but can't freeze the loop. At start-up we fill the filters with the first reading, so the angle starts out right instead of slowly climbing from 0.

Register	Value	What it does
0x6B PWR_MGMT_1	0x00	Wake up
0x19 SMPLRT_DIV	7	8 kHz / 8 = 1 kHz sample rate, same as our loop
0x1B GYRO_CONFIG	0x00	±250 °/s (131 counts per °/s)
0x1C ACCEL_CONFIG	0x00	±2 g (16,384 counts per g)
0x37 / 0x38	0x10 / 0x01	Data-ready interrupt on (we don't use the pin; the PWM ISR polls instead)
0x3B / 0x43	read 6 bytes each	Accelerometer x, y, z and gyro x, y, z

Table 3. MPU-6050 registers used by `mpu6050.c`.

3.4 The control loop

Everything time-critical happens in `on_pwm_wrap()` on core 0, once every millisecond:

1. Clear the interrupt and toggle GPIO 14 (so we can check the timing on a scope).
2. Read the IMU.
3. Smooth a_x and a_z and get the accelerometer angle with `atan2`.
4. Update the complementary filter with gyro y.
5. Check the demo button and step the target if it's time (Section 3.6).
6. If the loop is on, run PID; if not, reset the I term.
7. Update the PWM level if it changed.
8. Every 10th time, save the angle and duty for the screen.

Here's the controller part, straight from `imu_demo.c` (comments shortened):

```
angle_error = desired_angle - complementary_angle ;              // + means arm is below target
long long p_term = ((long long)Kp) * ((long long)angle_error) ;
long long d_term = ((long long)Kd) * ((long long)gyro[1]) ;      // de/dt = -gyro y
fix15 i_step = multfix15(Ki, angle_error) ;
int pd = (int)((p_term - d_term) >> 32) ;                         // 15.16 x 15.16 -> integer counts
int u_limit = control + DUTY_SLEW ;                                // overcurrent protection: slew limit
if (u_limit > DUTY_MAX) u_limit = DUTY_MAX ;
if (!(i_step > 0 && pd + fix2int15(i_term) >= u_limit)) i_term += i_step ;  // conditional integration
if (i_term > int2fix15(i_max)) i_term = int2fix15(i_max) ;
if (i_term < 0) i_term = 0 ;
int u = pd + fix2int15(i_term) ;
if (u < 0) u = 0 ;
if (u > u_limit) u = u_limit ;
control = u ;
```

Positive error means the arm is too low and needs more thrust. D is subtracted because a fast-rising arm should get *less* thrust, and a fast-falling arm should get more.

3.5 Overcurrent protection and windup

When the target jumps, the P term jumps too ($80 \times 90° = 7200$ counts when the arm is let go from hanging). Slamming a stopped motor from 0 to full voltage pulls a big surge of current, because it isn't spinning yet so nothing limits the current except the coil's resistance. So the final code lets the command *go up* by at most 20 counts per millisecond. Going down isn't limited, so the controller can still cut thrust instantly.

$$u_{lim,\,n} = \min(u_{n-1} + 20,\ 6000) \qquad \Rightarrow \qquad 0 \to 6000 \text{ counts in } 300 \text{ ms}$$

You can see this ramp in every step in Section 5.2, as a dotted line going up in the duty plot. 300 ms is short compared to the arm's roughly 1 s response, so it barely slows things down.

The ramp creates a new windup problem: while the output is stuck at the ramp limit, the error stays big and I would keep growing. So we only let I grow when the output isn't already at the limit. This is the "clamping" fix from the course PID notes, and it's why `i_max` could stay at 10000 (above the 6000 max output) and never kick in. I is also kept at 0 or above, since negative I would only delay thrust.

3.6 Demo button

The button is on GPIO 15, wired to ground with the internal pull-up (pressed = low). The ISR checks it every 1 ms and only believes a change after it's held for 20 checks in a row (20 ms debounce). Since each ISR call is 1 ms, we time the demo by counting calls:

When	What happens
Button held	Loop off, motor off, I reset, target –90°: the arm hangs straight down
Released, t = 0	Loop on, target 0° (level)
t = 5 s	Target +30°
t = 10 s	Target –30°
t = 15 s	Target 0°; the sequence ends and the arm stays level

Table 4. Demo sequence. It uses whatever gains are loaded, with no reset needed.

The target only gets set at those four moments, so a serial `a` command can still change it in between.

3.7 VGA display and serial commands

The Pico drives the monitor through the adapter board we soldered in Lab 2 (Figure 10). We reused the course's VGA driver (Figure 11): three PIO state machines make the sync and colour signals, and DMA streams the pixels to them, so the CPU doesn't have to. Core 1 does nothing but draw, so drawing can never slow down the control loop.

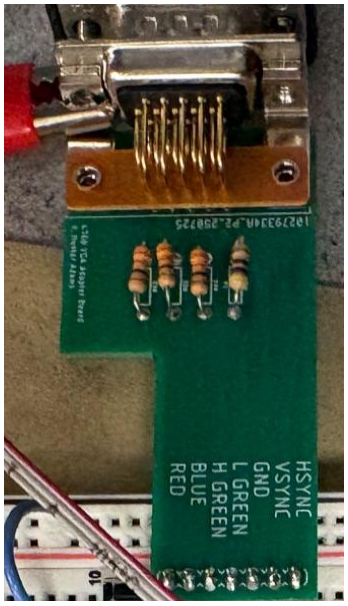


Figure 10. Our VGA adapter board (from Lab 2).

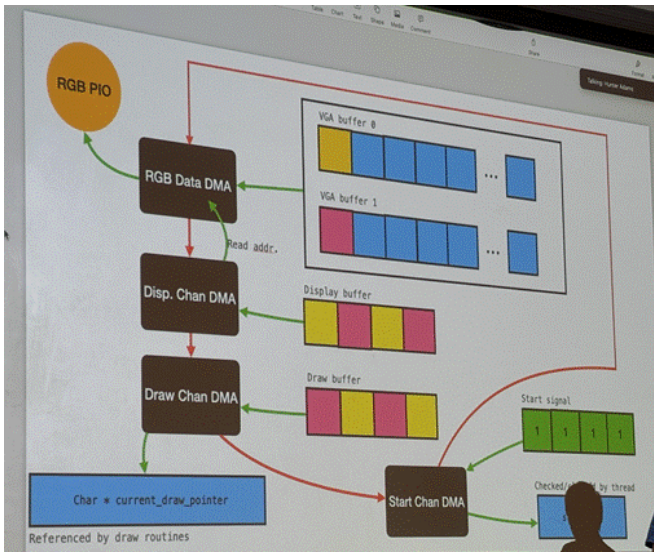


Figure 11. The VGA driver's DMA chain, from lecture.

- **Top plot:** the filtered angle (yellow) and the target (light blue), −90° to +90°.
- **Bottom plot:** the motor command `control`, 0 to 6000.
- **Text:** both angles, target, the three gains, duty, mode, I max and the live I term.

One point is saved every 10 ms, so the screen shows the last 5.3 s. Note: we plot the raw controller output, not a smoothed copy like the handout asks for. With D = 12 and the ramp limit it was smooth enough to

read (Section 5.2), and adding the course's one-line smoothing filter (`motor_disp`) is on our "would change" list (Section 6).

Serial commands. Type a letter, then a value. The value goes straight into the variables the ISR reads, so it takes effect within 1 ms.

Cmd	Sets	Units / range
`a`	Target angle	–90 to +90° (0 = level)
`p`	K_p	duty counts per degree, 0–6000
`i`	K_i	duty counts per degree per ms, 0–100 (0 also resets I)
`d`	K_d	duty counts per °/s, 0–6000
`m`	I cap `i_max`	0–6000 duty counts
`c`	Loop on / off	1 / 0 (0 also turns the motor off)
`n`	Manual duty (turns the loop off)	0–6000
`t`	Plot speed	ISR calls per plotted point (default 10)

Table 5. Serial commands.

3.8 Wiring

Signal	Pico pin	Notes
VGA Hsync / Vsync	GPIO 16 / 17	PIO state machines
VGA Green lo / Green hi / Blue / Red	GPIO 18 / 19 / 20 / 21	470 Ω / 330 Ω / 330 Ω / 330 Ω
MPU-6050 SDA / SCL	GPIO 8 / 9	I2C0, 400 kHz; IMU powered from 3.3 V
Motor PWM	GPIO 4	Slice 2 channel A, to the 4N35 through 330 Ω
(PWM slice 2 B)	GPIO 5	Set to 0, unused
ISR timing	GPIO 14	Toggles every ISR (for the scope)
Demo button	GPIO 15	To GND, internal pull-up, 20 ms debounce

Table 6. Pico wiring.

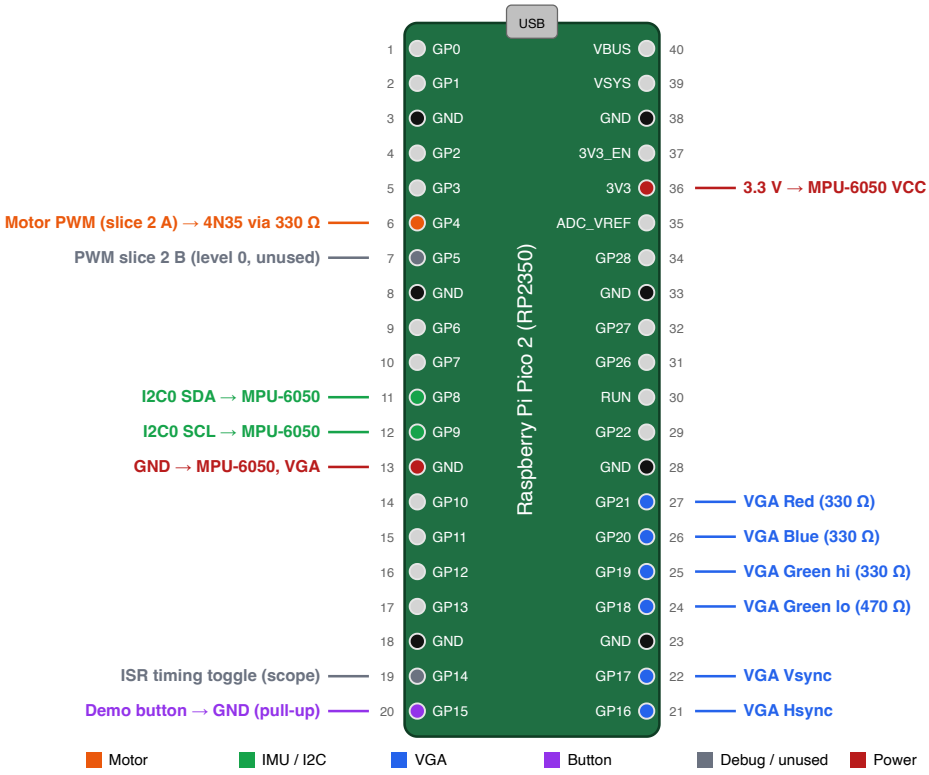


Figure 12. Pico 2 pinout with our pins, coloured by subsystem.

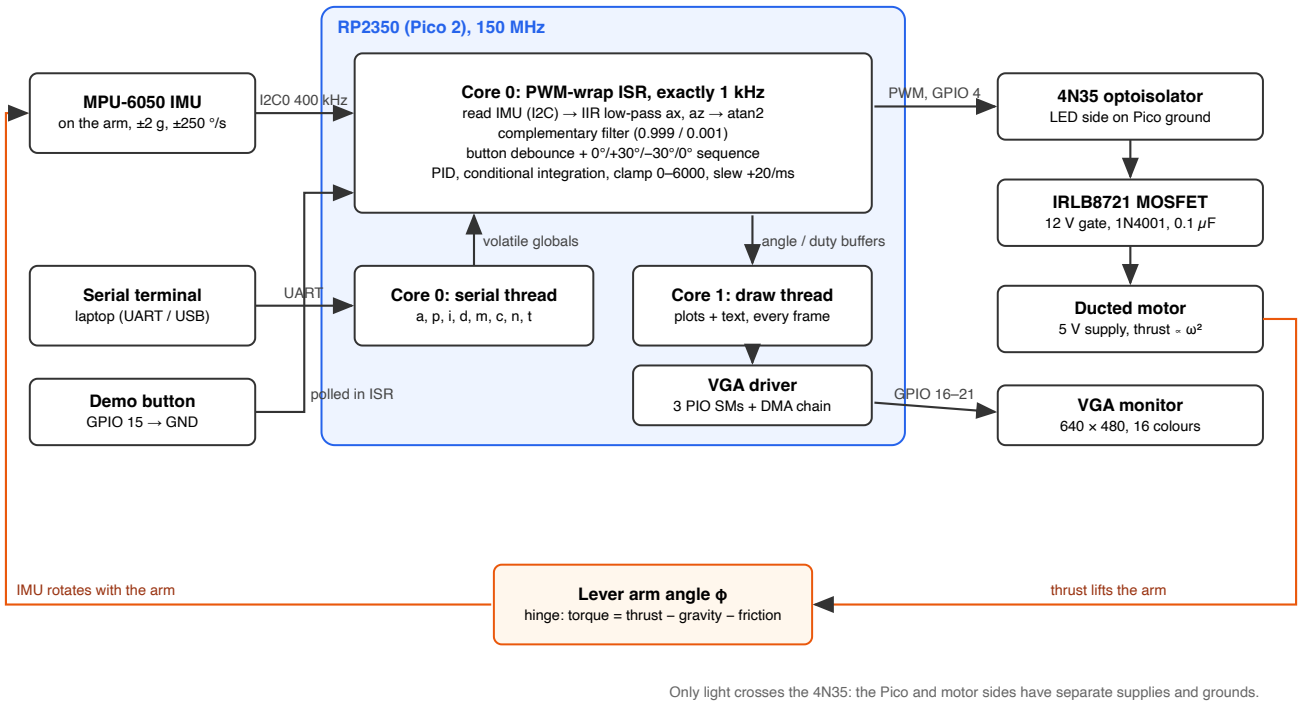


Figure 13. System block diagram. Core 0 runs the 1 kHz control loop and the serial thread; core 1 only draws. The orange path is the physical feedback through the arm.

4 Documentation

Where to find the drawings, code and data the course policy asks for:

Item	Where
Motor circuit schematic	Figure 9, Section 3.2
System block diagram	Figure 13
Pinout and wiring	Figure 12, Table 6
Complementary filter	Figure 2, Figure 3, Section 2.2
IMU registers, serial commands, demo timing	Table 3, Table 5, Table 4
VGA adapter and driver	Figure 10, Figure 11
Photos of the rig	Figure 1 and the photos in Figure 14–Figure 17
VGA screenshots (demo)	Figure 14–Figure 17; earlier runs in Figure 18, Figure 19
Settling times, hover duties	Table 8, Table 9
Commented code	Appendix A
AI prompt log	Appendix B

Table 7. Documentation index.

5 Results

5.1 Final gains

$K_p = 80$, $K_i = 0.2$, $K_d = 12$, `i_max` = 10000 (never reached), output 0–6000 with a ramp limit of +20 counts per ms. One set of gains ran the whole demo with no reset. Section 7 (Q3) explains how we picked them.

5.2 Demo sequence

Figure 14–Figure 17 show the four steps of the demo, taken from our video of the checked-off run. Each VGA screen (left) is paired with a photo of the arm at that angle (right). On every screen, the top plot is the angle (yellow) vs. the target (blue), the bottom plot is the motor command (0–6000), and the newest data is on the right.

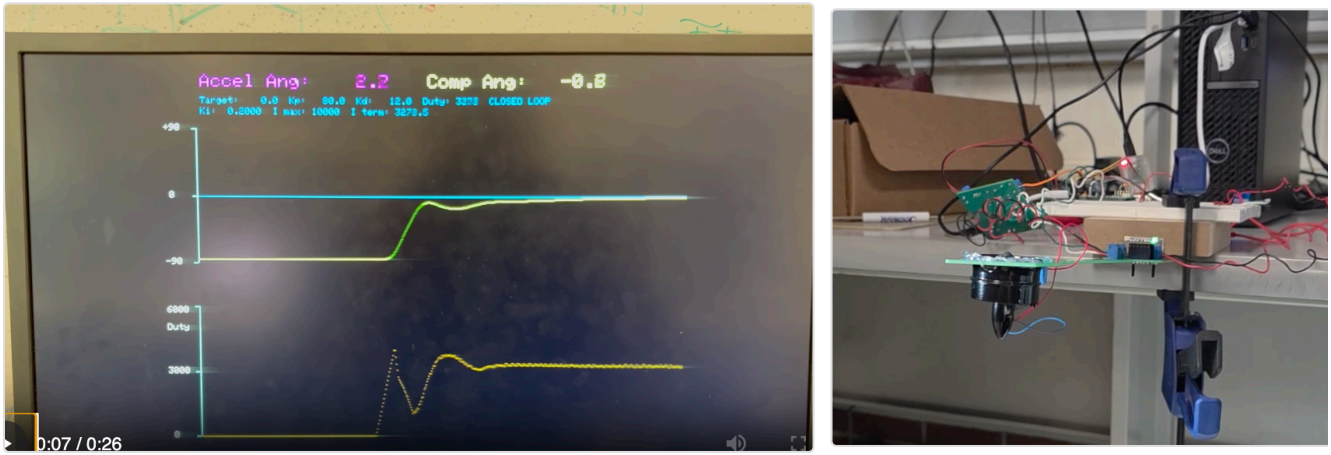
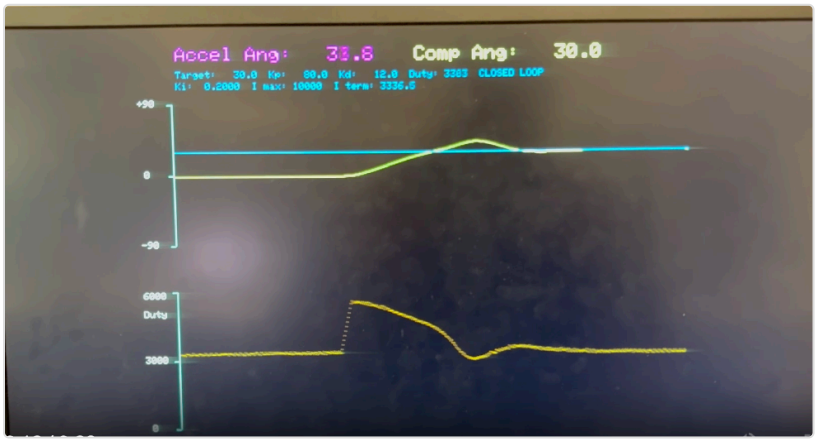


Figure 14. t = 0: let go, –90° → 0°. The duty starts at 0 (motor off while the button is held), ramps up (the dotted rise) to about 4100, dips to about 1300 as D brakes the rising arm, then settles at 3278. No overshoot; I pulls in the last few degrees. At the end: angle –0.8°, duty 3278, I term 3278.5, so I is basically the whole command, as Section 2.4 says. Right: the arm level (seen edge-on), fan hanging below.



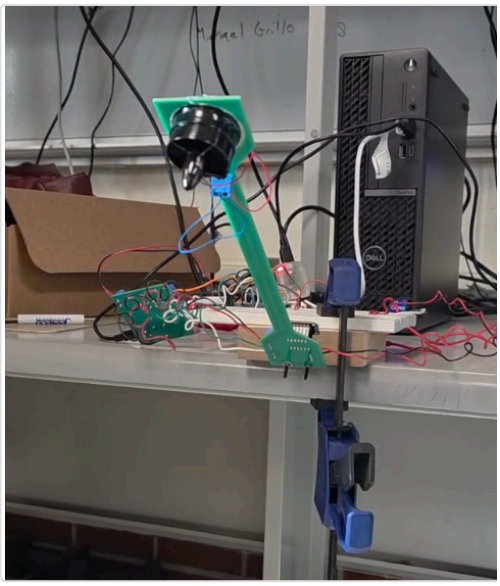
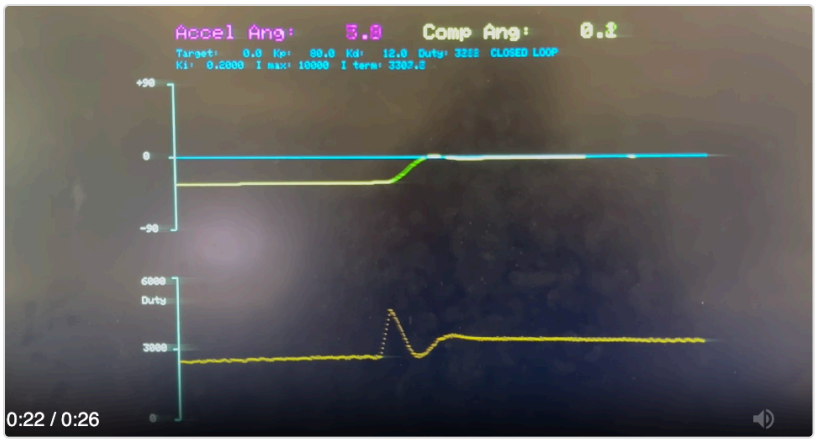


Figure 15. t = 5 s: 0° → +30°. P kicks in (80 × 30° = 2400 extra counts) and the duty ramps up to about 5800. The arm overshoots to about +46°, the duty dips to about 3100 while the extra I built up on the way up drains off, and then it settles at +30.0° with duty 3383 and I term 3336.5. Right: the arm at +30°.





Figure 16. t = 10 s: +30° → –30°. The command drops straight to 0. The fan can't pull down, so gravity does the work. The arm falls past the target, P and D ask for about 5000 counts to catch it, and it still dips to about –56° and bounces once. I drained during the fall, so the end is a slow creep: duty 2533 and still climbing (I term 2433.9), arm at –32.8°. Right: the arm at –30°.



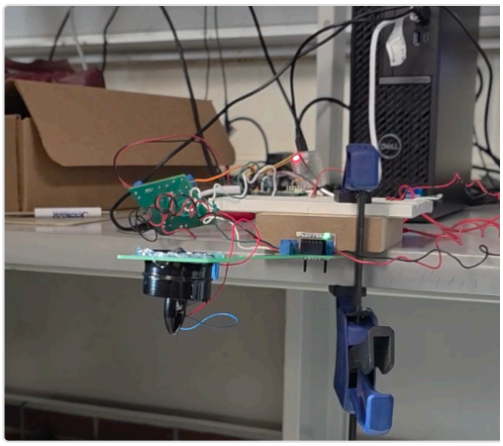


Figure 17. t = 15 s: –30° → 0°. The fastest step. I is already close to what it needs for level, so a short P kick (duty about 4700) lifts the arm with only about 4° of overshoot. It settles at 0.2° with duty 3288 and I term 3307.2. Right: the arm back at level (same pose as Figure 14).

5.3 Settling times

Step	Settling time (s)	Overshoot (from screen)	What slowed it down
–90° → 0°	2.74	none (comes up from below)	I starts at 0 and has to build up the hover command
0° → +30°	2.38	≈ 16° (peak ≈ +46°)	Extra I from the climb has to drain off
+30° → –30°	3.56	≈ 26° (low ≈ –56°)	Gravity-driven fall (fan can't pull down); I has to rebuild
–30° → 0°	0.98	≈ 4°	I already close to the right value

Table 8. Settling times measured from our video of the demo. Overshoot was read off the VGA screenshots (1.2° per pixel, so about ±3°).

The pattern makes sense. Going up, the fan pushes and gravity and D brake, so it's well controlled. Going down, gravity does the pushing and only the fan can brake, and it has to ramp back up from zero first, so that step overshoots the most and takes the longest. In every step, how long the tail lasts depends on how far the I term has to move to reach the new hover value.

5.4 Hover duty vs. angle

Once the arm is settled, P and D are about zero, so the duty tells us how much thrust each angle needs.

Angle	Duty (I term)	Duty as % of period	$(\text{duty} / \text{duty}_{0°})^2 \approx$ thrust ratio	Simple-arm prediction cos ϕ
0°	3278 (3279)	54.6%	1.00	1.00
+30°	3383 (3337)	56.4%	1.07	0.87
–30°	2533 (2434), still rising	42.2%	≤ 0.60	0.87

Table 9. Settled duty at each demo angle, from the VGA readouts. The thrust ratio assumes thrust $\propto \omega^2$ and fan speed roughly $\propto$ average voltage.

A simple arm would need the same thrust at ±30°. Ours needs *more* at +30° and a lot less at −30°, because the fan hangs below the arm (Section 2.3). Fitting our numbers gives d/r ≈ 0.4–0.5. That's rough (the −30° duty was still creeping up), but the direction is clear. It also puts the arm's tipping point around 20–30° above level instead of exactly at level.

5.5 Stable range

With the final gains, the arm was stable from **−90°** (hanging; the controller just turns the fan off) up to about **+50°**. The top limit changed depending on which 5 V motor supply we used; above it, the arm wasn't stable with these gains (more in Section 7, Q4).

5.6 Earlier tuning runs

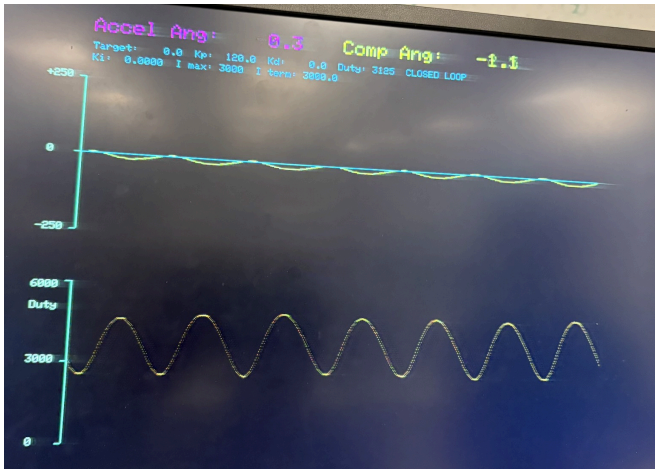


Figure 18. Early P-only test (K_p = 120, no I or D, target 0°) on an older build with a ±250 angle axis. The arm oscillates steadily, about 6.5 cycles in 5.3 s (≈ 1.2 Hz): right at the edge of stable. The I term still showed 3000 from an earlier run, because back then setting K_i = 0 didn't reset it. The final code does reset it.

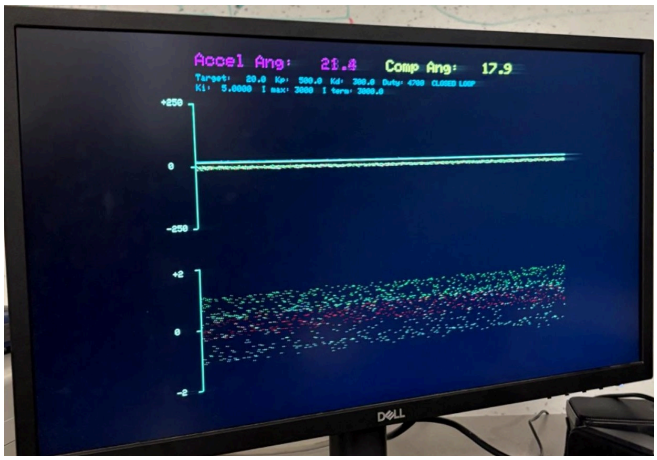


Figure 19. Early build with much bigger gains (K_p = 500, K_d = 300, K_i = 5). The bottom plot showed the raw accelerometer (±2 g): with the fan on, vibration spreads it across most of the range, while the filtered angle on top is a clean line. This is why we need the complementary filter.

6 Conclusion

Our helicopter did everything the lab asked for. It ran the full 0° / +30° / −30° / 0° demo on one set of gains with no reset, every step settled within 3.6 s, we could change the gains and target from the serial terminal at any time, and the VGA showed the angle, target and motor command live.

The results also matched the theory. The complementary filter kept fan vibration out of the angle. Once settled, the I term equalled the hover duty, like PID theory says it should. And the lopsided settling times and hover duties are what you'd expect from a fan that can only push up and an arm whose weight hangs off-centre.

What we'd change next time:

- Plot a smoothed copy of the motor command (`motor_disp`) like the handout asks, instead of the raw one.
- Measure the gyro's built-in error at start-up (average a second of readings with the arm still) and subtract it.
- Add a "feed-forward" guess of the hover thrust using the r cos φ + d sin φ model, so I only has to fix what's left. That should shorten the slow tails, especially after going down.
- Move the `printf` for button presses out of the ISR, since printing inside the 1 kHz interrupt can make one loop run long.
- Measure the arm's length and mass so we can check the model with real numbers.

7 Answers to the Lab Report Questions

The helicopter handout asks the report to include the following.

Q1. A schematic of the circuit you built

See Figure 9, plus Table 6, Figure 12 and Figure 13. GPIO 4 (PWM) lights the 4N35's LED through 330 Ω. The 4N35 switches the 12 V gate supply (through a kill switch) onto the IRLB8721 MOSFET's gate, with a 10 kΩ pull-down on the gate and a 1 MΩ resistor to help it switch off fast. The motor runs from a separate 5 V supply with a 1N4001 diode and a 0.1 μF capacitor across it. The Pico and motor sides share no ground. The IMU (GPIO 8/9), VGA (GPIO 16–21) and button (GPIO 15) are wired as in Table 6.

Q2. VGA screen dumps of (1) motor control and (2) actual angle, including the four-angle demo

See Figure 14–Figure 17, one for each demo step (0°, +30°, −30°, 0°), each next to a photo of the arm at that angle. The top trace is the angle vs. the target; the bottom trace is the motor command (0–6000). While the button is held, the command is 0 and the arm hangs at −90° (left edge of Figure 14). Every step shows the ramp-limited rise in the command, and the settled command matches the I term.

Q3. How you selected the three PID gains

Ballpark from units. We picked starting values so each term would give a reasonable duty count:

- K_p: a 90° error (hanging, target level) should ask for about full power, so $K_p \approx 6000/90 \approx 67$. So: tens.
- K_d: the gyro rate in °/s is about the same size as the error in degrees, so K_d is also tens. (The course page's "thousands" assumes taking differences of noisy samples, not a gyro.)
- K_i: I adds $K_i \cdot e$ a thousand times a second, so K_i has to be much smaller. We started near 1/32.

Tuning steps (the order from lecture and the PID notes):

1. **P only:** turn up K_p until you find the edge between "wobbles die out" and "wobbles grow." $K_p = 120$ oscillated steadily at about 1.2 Hz (Figure 18), so we backed off to 80, about two-thirds of that.
2. **Add D** until the wobbling stops. Our test: push the arm down below the target and let go. It should spring back, overshoot once, and settle quickly. We ended at $K_d = 12$.
3. **Add I** to get rid of the leftover error and bring the arm onto the target. More I gets there faster but overshoots and wobbles slowly. We settled on $K_i = 0.2$. At that rate a 5° error grows I by 1000 counts per second, enough to build the roughly 3300-count hover command in a few seconds.
4. **Run the whole demo** with the button and adjust. The windup fix (Section 3.5) handled the rest, so we left `i_max` at 10000.

The live I-term readout on the VGA was our most useful tuning tool: once the arm settles, it should equal the duty, which tells you P and D are done and only I is holding the arm up (Figure 14–Figure 17).

Q4. What are the maximum/minimum angles that produce stable behaviour for your PID gains?

Max ≈ +50°, min −90° (0° = level). The max depended on which 5 V motor supply we used. Why there's a limit:

- **Min:** −90° is just hanging. Near there the thrust needed drops to zero, the output sits at 0 and gravity holds the arm, so it's stable for free.
- **Max:** above the arm's tipping point (level for a simple arm, a bit higher for ours, Section 5.4), gravity pushes the arm further away instead of back. The higher you go, the stronger that push, until D can't keep up.
- **The plant changes with angle:** how much the arm responds to one duty count depends on fan speed, and our gains were tuned for the demo range near level.
- **The supply matters:** a different supply changes thrust per duty count (thrust $\propto V^2$), which scales the whole loop. The same gains end up closer to or further from the edge, so the max angle moved.

Q5. What are the settling times at different angles?

Step	Settling time	Overshoot
−90° → 0° (let go)	2.74 s	none
0° → +30°	2.38 s	≈ 16°
+30° → −30°	3.56 s	≈ 26°
−30° → 0°	0.98 s	≈ 4°

(Same data as Table 8.) Going up takes about 1–2.7 s. Going down is slowest (3.56 s) because the fan can only cut thrust: gravity drives the fall, the arm dips past the target, and I has to rebuild. The fastest step started with I already near the right value; the slowest upward step (letting go from −90°) started with I at zero.

Q6. A heavily commented listing of your code

Appendix A lists `imu_demo.c`, `mpu6050.c`, `mpu6050.h` and `CMakeLists.txt` exactly as we demoed them. The course VGA driver and protothreads header weren't changed, so they aren't listed. Every change from the course MPU-6050 demo is marked in the comments (`ADDED`, `CHANGED`, `FIXED`, `REMOVED`) and shaded in the listing.

Q7. A prompt log from any AI tool used for code generation

Appendix B has the full log of our coding sessions with Claude Code (model: Claude Opus 5.5): 18 prompts from Oct 21–22, 2026, with our prompts word for word and Claude's responses. Totals: 401 prompt words, 1719 response words and 214 lines of code given. Suggested additions / deletions / changes: **36 / 1 / 22**. Accepted (still in the final code): **29 / 1 / 19**.

8 Work Distribution and AI Use

Amber, Ivan, and Zarif worked together on all parts of the lab. We often met outside of our lab section, and most of the work was done together in the lab. Outside the lab, we kept in touch so everyone knew what was done and what was next.

AI use. We used Claude Code (Claude Opus 5.5) while writing the program. Every prompt and response is in Appendix B, and the code it contributed is marked with `ADDED` / `CHANGED` / `FIXED` comments (shaded in Appendix A). We also used Claude to help organise, typeset and edit this report from our notes, draft report, photos, screenshots and measurements. Code it suggested while writing the report (a smoothed display variable, a different button routine on GPIO 2, and extra comments) was not added to our program. All measurements, photos, screenshots and hardware are ours.

References

1. V. H. Adams, *PID control of a 1D helicopter* (lab handout). https://vanhunteradams.com/Pico/Helicopter/Helicopter.html
2. V. H. Adams, *Complementary filters*. https://vanhunteradams.com/Pico/ReactionWheel/Complementary_Filters.html
3. V. H. Adams, *Phenomenological introduction to PID controllers*. https://vanhunteradams.com/PID/PID.html
4. V. H. Adams, *Analytical introduction to PID controllers*. https://vanhunteradams.com/PID/PID_Analytical.html
5. V. H. Adams, *DC motor circuit and PWM*. https://vanhunteradams.com/Pico/Helicopter/MotorCircuit.html
6. V. H. Adams, *Data display*. https://vanhunteradams.com/Pico/Helicopter/Display.html
7. V. H. Adams, *Inter-Integrated Circuit (I2C)*. https://vanhunteradams.com/Protocols/I2C/I2C.html
8. V. H. Adams, MPU6050 IMU demo and PWM demo code. https://github.com/vha3/Hunter-Adams-RP2040-Demos
9. Raspberry Pi Ltd., *RP2350 Datasheet* (PWM and I2C chapters).
10. InvenSense, *MPU-6000/MPU-6050 Register Map and Descriptions*.
11. B. Land, *ECE 4760/5730 course policy* (lab report sections). https://people.ece.cornell.edu/land/courses/ece4760/EE4760_policy.html

Appendix A Commented Code Listing

These are the files exactly as we built and demoed them (Pico 2 / RP2350, Pico SDK 2.3.1). The program also uses the course's VGA driver (`VGA/vga16_graphics_v3.c/.h` and its three PIO programs) and `pt_cornell_rp2040_v1_4.h`, unchanged. Changes from the course MPU-6050 demo are marked in the comments with `ADDED`, `CHANGED`, `FIXED` or `REMOVED`. Those are the parts written with AI help (Appendix B) and later edited by us. ▢ marks those lines. Comments are green.

A.1 imu_demo.c (main program: ISR, controller, display, serial)

```
/**
 * V. Hunter Adams (vha3@cornell.edu)
 *
 * This demonstration utilizes the MPU6050.
 * It gathers raw accelerometer/gyro measurements, scales
 * them, and plots them to the VGA display. The top plot
 * shows the target and actual (complementary) angle, bottom
 * plot shows the fan duty cycle (PID output).
 * CHANGED: top plot was gyro, bottom plot was accelerometer.
 *
 * HARDWARE CONNECTIONS
 *  - GPIO 16 ---> VGA Hsync
 *  - GPIO 17 ---> VGA Vsync
 *  - GPIO 18 ---> 470 ohm resistor ---> VGA Green
 *  - GPIO 19 ---> 330 ohm resistor ---> VGA Green
 *  - GPIO 20 ---> 330 ohm resistor ---> VGA Blue
 *  - GPIO 21 ---> 330 ohm resistor ---> VGA Red
 *  - RP2040 GND ---> VGA GND
 *  - GPIO 8 ---> MPU6050 SDA
 *  - GPIO 9 ---> MPU6050 SCL
 *  - 3.3v ---> MPU6050 VCC
 *  - RP2040 GND ---> MPU6050 GND
 *  - GPIO 4 ---> PWM_OUT
 *  - GPIO 15 ---> pushbutton ---> GND (internal pull-up)
 */


// Include standard libraries
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
// Include PICO libraries
#include "pico/stdlib.h"
#include "pico/multicore.h"
// Include hardware libraries
#include "hardware/pwm.h"
#include "hardware/dma.h"
#include "hardware/irq.h"
#include "hardware/adc.h"
#include "hardware/pio.h"
#include "hardware/i2c.h"
#include "hardware/clocks.h"
#include "hardware/sync.h"
// Include custom libraries
#include "VGA/vga16_graphics_v3.h"
#include "mpu6050.h"
#include "pt_cornell_rp2040_v1_4.h"

// We will start drawing at column 81
#define LEFT_EDGE 81

// Rescale the measurements for display
#define OLD_RANGE 500.  // (+/- 250)
#define NEW_RANGE 150.  // (looks nice on VGA)
```

```
#define OLD_MIN -250.
#define OLD_MAX 250.
#define ACCEL_SCALE 120.0

// Angle plot range (degrees)
#define ANGLE_MIN -90.
#define ANGLE_RANGE 180.  // (+/- 90)
#define RAD2DEG 57.2957795f

// Place the plots
#define TOP_PLOT_BOTTOM 230
#define BOTTOM_PLOT_BOTTOM 430
#define PLOT_WIDTH 530

// Arrays in which raw measurements will be stored
fix15 acceleration[3], gyro[3];

// Arrays for circular display buffer
fix15 ax[PLOT_WIDTH] ;
fix15 ay[PLOT_WIDTH] ;
fix15 az[PLOT_WIDTH] ;
fix15 gx[PLOT_WIDTH] ;
fix15 gy[PLOT_WIDTH] ;
fix15 gz[PLOT_WIDTH] ;

// ===== ADDED: complementary filter =====

// Low-passed accelerometer measurements (15.16 fixed point, g's)
fix15 filtered_ax, filtered_az ;

// Angle estimates (15.16 fixed point, degrees)
fix15 accel_angle ;          // angle from accelerometer only (noisy, no drift)
fix15 gyro_angle_delta ;     // angle change from gyro this timestep
fix15 complementary_angle ;  // fused angle estimate

// Circular display buffers for the angle estimates
fix15 accel_angle_buf[PLOT_WIDTH] ;
fix15 comp_angle_buf[PLOT_WIDTH] ;
// ===== END ADDED =====

// ===== ADDED: circular display buffer for the fan duty cycle (PID output) =====
int duty_buf[PLOT_WIDTH] ;
// ===== END ADDED =====

// Pointer to head of buffer
volatile int head = 0 ;

// character array
char screentext[80];   // CHANGED: was 40, controller readout line is longer

// draw speed
volatile int threshold = 10 ;

// Some macros for max/min/abs
#define min(a,b) ((a<b) ? a:b)
#define max(a,b) ((a<b) ? b:a)
#define abs(a) ((a>0) ? a:-a)

// semaphore
static struct pt_sem vga_semaphore ;

// Angle player button
#define BUTTON_PIN 15

// ===== ADDED: button angle sequence =====
// Hold button: target -90, fan off. Release: closed loop, target 0 -> 30 (t=5s) -> -30 (t=10s) -> 0 (t=15s)
#define DEBOUNCE_MS 20                          // button must be stable this long to count
int btn_held = 0 ;                              // debounced button state (1 = held)
int btn_count = 0 ;                             // ms the raw reading has disagreed with btn_held
```

```
int seq_active = 0 ;                            // 1 = running the timed angle sequence
int seq_ms = 0 ;                                // ms since the button was released
// ===== END ADDED =====

// Some paramters for PWM
#define WRAPVAL 6000
#define CLKDIV  25.0
// ADDED: max allowed fan duty cycle (closed loop and manual 'p'), below WRAPVAL
#define DUTY_MAX 6000
// ADDED: max duty increase per 1 ms sample in closed loop (inrush/overcurrent protection).
// 20 counts/ms -> 0 to DUTY_MAX in 300 ms. Decreases are not limited.
#define DUTY_SLEW 20
// GPIO we're using for PWM
#define PWM_OUT 4
uint slice_num ;
// PWM duty cycle
volatile int control ;
volatile int old_control ;

// ===== ADDED: proportional controller =====
volatile int closed_loop = 0 ;                  // 0 = manual duty ('p'), 1 = P control
volatile fix15 desired_angle = 0 ;              // target angle (degrees, 15.16)
volatile fix15 Kp = int2fix15(0) ;             // gain (duty counts per degree, 15.16)
fix15 angle_error ;                             // desired - measured (degrees, 15.16)
// ===== END ADDED =====

// ===== ADDED: derivative control =====
volatile fix15 Kd = int2fix15(0) ;              // gain (duty counts per deg/s, 15.16), 0 = off
// ===== END ADDED =====

// ===== ADDED: integral control =====
volatile fix15 Ki = int2fix15(0) ;              // gain (duty counts per deg, per 1 ms sample, 15.16), 0
= off
volatile int i_max = 10000 ;                     // anti-windup cap on the I term (duty counts)
volatile fix15 i_term = 0 ;                     // accumulated Ki*error (duty counts, 15.16)
// ===== END ADDED =====


void drawAxes() {
    // Draw the static aspects of the display
    setTextSize(1) ;
    setTextColor(WHITE);

    // Draw bottom plot
    drawHLine(75, 430, 5, CYAN) ;
    drawHLine(75, 355, 5, CYAN) ;
    drawHLine(75, 280, 5, CYAN) ;
    drawVLine(80, 280, 150, CYAN) ;
    // CHANGED: labels are now duty cycle (0 to DUTY_MAX), were accel (-2 to +2 g)
    sprintf(screentext, "%d", DUTY_MAX/2) ;
    setCursor(45, 350) ;
    writeString(screentext) ;
    sprintf(screentext, "%d", DUTY_MAX) ;
    setCursor(45, 280) ;
    writeString(screentext) ;
    sprintf(screentext, "0") ;
    setCursor(50, 425) ;
    writeString(screentext) ;
    sprintf(screentext, "Duty") ;
    setCursor(45, 300) ;
    writeString(screentext) ;

    // Draw top plot
    drawHLine(75, 230, 5, CYAN) ;
    drawHLine(75, 155, 5, CYAN) ;
    drawHLine(75, 80, 5, CYAN) ;
    drawVLine(80, 80, 150, CYAN) ;
    sprintf(screentext, "0") ;
    setCursor(50, 150) ;
    writeString(screentext) ;
```

```
    sprintf(screentext, "+90") ;
    setCursor(45, 75) ;
    writeString(screentext) ;
    sprintf(screentext, "-90") ;
    setCursor(45, 225) ;
    writeString(screentext) ;
}

volatile int throttle = 10 ;

// Interrupt service routine
void on_pwm_wrap() {

    // Clear the interrupt flag that brought us here
    pwm_clear_irq(pwm_gpio_to_slice_num(5));
    gpio_put(14, !gpio_get(14)) ;

    // Read the IMU
    // NOTE! This is in 15.16 fixed point. Accel in g's, gyro in deg/s
    // If you want these values in floating point, call fix2float15() on
    // the raw measurements.
    mpu6050_read_raw(acceleration, gyro);

    // ===== ADDED: low pass filter on accelerometer (first-order IIR, alpha = 1/16) =====
    filtered_ax = filtered_ax + ((acceleration[0] - filtered_ax) >> 4);
    filtered_az = filtered_az + ((acceleration[2] - filtered_az) >> 4);
    // ===== END ADDED =====

    // Accelerometer angle (degrees - 15.16 fixed point)
    // NO SMALL ANGLE APPROXIMATION --> ADDED LOW PASS FILTER
    // IMU rotates about +y axis
    // FIXED: removed +PI so flat (arm horizontal) = 0 deg, range -180 to +180
    accel_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_az)), oneeightyoverpi);

    // Gyro angle delta (measurement times timestep) (15.16 fixed point)
    // gyro y for rotation about +y axis
    gyro_angle_delta = multfix15(gyro[1], zeropt001) ;

    // Complementary angle (degrees - 15.16 fixed point)
    // gyro and accel share the same sign convention so + instead between (comp ang + gyro ang)
    complementary_angle = multfix15(complementary_angle + gyro_angle_delta, zeropt999) +
multfix15(accel_angle, zeropt001);

    // ===== ADDED: button angle sequence (ISR runs at ~1 kHz, so one call = 1 ms) =====
    // Button is active low (pull-up). Debounce: accept a new state only after it holds DEBOUNCE_MS.
    int pressed = !gpio_get(BUTTON_PIN) ;
    if (pressed != btn_held) {
        if (++btn_count >= DEBOUNCE_MS) {
            printf("Button %s\n", pressed ? "pressed" : "released");
            btn_held = pressed ;
            btn_count = 0 ;
            // Released: start the sequence under closed loop control
            if (!btn_held) {
                seq_active = 1 ;
                seq_ms = 0 ;
                closed_loop = 1 ;
            }
        }
    }
    else {
        btn_count = 0 ;
    }
    if (btn_held) {
        // Held: arm rests at -90, fan off (closed_loop = 0 also clears the I term below)
        closed_loop = 0 ;
        control = 0 ;
        desired_angle = int2fix15(-90) ;
        seq_active = 0 ;
    }
    else if (seq_active) {
```

```
263         // Only write the target at each step so serial 'a' can still override in between
264         if      (seq_ms == 0)     desired_angle = int2fix15(0) ;
265         else if (seq_ms == 5000)  desired_angle = int2fix15(30) ;
266         else if (seq_ms == 10000) desired_angle = int2fix15(-30) ;
267         else if (seq_ms == 15000) {
268             desired_angle = int2fix15(0) ;
269             seq_active = 0 ;   // sequence done, stay at 0
270         }
271         seq_ms++ ;
272     }
273     // ===== END ADDED =====
274
275     // ===== ADDED: proportional control =====
276     // ===== CHANGED: now PID control (derivative and integral terms added below) =====
277     if (closed_loop) {
278         // Positive error = arm is below the target, so more thrust is needed
279         angle_error = desired_angle - complementary_angle ;
280         // Kp * error in 64 bits (avoids overflow), >>32 converts 15.16 * 15.16 to an integer duty cycle
281         long long p_term = ((long long)Kp) * ((long long)angle_error) ;
282         // ===== ADDED: derivative term =====
283         // d(error)/dt = -d(angle)/dt = -gyro[1] (gyro y is the arm's angular rate in deg/s).
284         // Using the gyro directly avoids the noise of differencing the angle estimate.
285         // Opposes motion: arm rising fast -> less thrust, arm falling fast -> more thrust.
286         long long d_term = ((long long)Kd) * ((long long)gyro[1]) ;
287         // ===== END ADDED =====
288         // ===== ADDED: integral term =====
289         // Accumulate Ki*error every sample (rather than summing error and scaling later),
290         // so the sum is already in duty counts and can't overflow 15.16, and changing Ki
291         // doesn't make the output jump. At steady state P and D are ~0, so the I term
292         // alone supplies the duty that holds the arm up against gravity.
293         fix15 i_step = multfix15(Ki, angle_error) ;
294         int pd = (int)((p_term - d_term) >> 32) ;
295         // ===== ADDED: overcurrent protection =====
296         // Highest duty allowed this sample: DUTY_MAX, and at most DUTY_SLEW above last sample,
297         // so a big setpoint jump ramps the motor up instead of stepping it (limits inrush current)
298         int u_limit = control + DUTY_SLEW ;
299         if (u_limit > DUTY_MAX) u_limit = DUTY_MAX ;
300         // Conditional integration: don't grow the I term while the output is already pinned at
301         // the limit, otherwise it winds up during the ramp/stall and overshoots afterwards
302         if (!(i_step > 0 && pd + fix2int15(i_term) >= u_limit)) i_term += i_step ;
303         // ===== END ADDED =====
304         // Anti-windup: cap at i_max (set it a bit above the duty that holds the arm at the
305         // target). Floor at 0 because the fan can't pull down, so negative I only adds lag.
306         if (i_term > int2fix15(i_max)) i_term = int2fix15(i_max) ;
307         if (i_term < 0) i_term = 0 ;
308         // ===== END ADDED =====
309         int u = pd + fix2int15(i_term) ;   // CHANGED: was Kp * error only, then P - D
310         // The fan can only push one way, so clamp to the valid duty range
311         if (u < 0) u = 0 ;
312         if (u > u_limit) u = u_limit ;   // CHANGED: was DUTY_MAX (and WRAPVAL before that), now also
    slew limited
313         control = u ;
314     }
315     // ===== ADDED: clear the integrator whenever the controller is off =====
316     // so it starts from 0 the next time closed loop is turned on
317     else {
318         i_term = 0 ;
319     }
320     // ===== END ADDED =====
321     // ===== END ADDED =====
322
323     // Update fan duty cycle
324     if (control!=old_control) {
325         old_control = control ;
326         pwm_set_chan_level(slice_num, PWM_CHAN_A, control);
327     }
328
329     // Increment drawspeed controller
330     throttle += 1 ;
331     // Maintain a circular buffer for display
```

```
    if ((throttle >= threshold)) {
        // Zero drawspeed controller
        throttle = 0 ;

        // Replace oldest data in accel arrays with new sample
        ax[head] = acceleration[0] ;
        ay[head] = acceleration[1] ;
        az[head] = acceleration[2] ;

        // Replace oldest data in gyro arrays with new sample
        gx[head] = gyro[0] ;
        gy[head] = gyro[1] ;
        gz[head] = gyro[2] ;

        // ===== ADDED: store complementary filter angle estimates for display =====
        accel_angle_buf[head] = accel_angle ;
        comp_angle_buf[head] = complementary_angle ;
        // ===== END ADDED =====

        // ===== ADDED: store fan duty cycle for display =====
        duty_buf[head] = control ;
        // ===== END ADDED =====

        // Move the pointer to the oldest data
        head = ((head+1) == PLOT_WIDTH) ? 0 : (head + 1) ;

    }
}

static PT_THREAD (protothread_draw(struct pt *pt))
{
    PT_BEGIN(pt) ;

    static int start ;
    static int i ;

    // Draw axes
    drawAxes() ;
    // Copy to other buffer
    copy_buffer_to_other() ;

    while(1) {
        // Wait for a buffer swap
        PT_YIELD_UNTIL(pt, draw_start_signal()) ;
        // REMOVED: gpio_put(15, ...) debug toggle, GPIO 15 is now the button

        // Clear the buffer, draw the axes
        clearLowFrame(0, BLACK) ;
        drawAxes() ;

        // Plot data oldest-->newest
        start = head ;
        for (i = 0; i < PLOT_WIDTH; i++) {
            // ===== CHANGED: bottom plot is now duty cycle (was ax/ay/az) =====
            // 0 duty at the bottom of the plot, DUTY_MAX at the top
            drawPixel(LEFT_EDGE + i, (BOTTOM_PLOT_BOTTOM -
                (int)(NEW_RANGE*((float)duty_buf[start]/DUTY_MAX))), ORANGE) ;
            // ===== END CHANGED =====

            // ===== CHANGED: top plot shows only the actual (complementary) angle =====
            // (gyro x/y/z and accel-only angle traces removed; target line drawn below)
            // Angles rotate about +y axis within the range ~[-90, 90] so 0 degree is when the IMU is
parallel to the floor
            drawPixel(LEFT_EDGE + i, (TOP_PLOT_BOTTOM -
                (int)(NEW_RANGE*((float)((fix2float15(comp_angle_buf[start]))-ANGLE_MIN)/ANGLE_RANGE))),
YELLOW) ;
            // ===== END CHANGED =====

            start = ((start+1)==PLOT_WIDTH) ? 0 : (start+1) ;
        }
```

```
400
401         // ===== ADDED: target angle line on top plot =====
402         drawHLine(LEFT_EDGE, (TOP_PLOT_BOTTOM -
403             (int)(NEW_RANGE*((float)((fix2float15(desired_angle))-ANGLE_MIN)/ANGLE_RANGE))), PLOT_WIDTH,
    LIGHT_BLUE) ;
404         // ===== END ADDED =====
405
406         // ===== ADDED: numeric readout of angle estimates =====
407         setTextSize(2) ;
408         setTextColor(MAGENTA) ;
409         setCursor(85, 20) ;
410         sprintf(screentext, "Accel Ang: %6.1f", fix2float15(accel_angle)) ;
411         writeString(screentext) ;
412         setTextColor(YELLOW) ;
413         setCursor(330, 20) ;
414         sprintf(screentext, "Comp Ang: %6.1f", fix2float15(complementary_angle)) ;
415         writeString(screentext) ;
416         setTextSize(1) ;
417         // ===== END ADDED =====
418
419         // ===== ADDED: controller readout =====
420         setTextColor(LIGHT_BLUE) ;
421         setCursor(85, 45) ;
422         // CHANGED: added Kd to the readout
423         sprintf(screentext, "Target: %6.1f  Kp: %6.1f  Kd: %6.1f  Duty: %4d  %s",
424             fix2float15(desired_angle), fix2float15(Kp), fix2float15(Kd), control, closed_loop ? "CLOSED
    LOOP" : "MANUAL") ;
425         writeString(screentext) ;
426         // ===== ADDED: integral readout (second line) =====
427         setCursor(85, 57) ;
428         sprintf(screentext, "Ki: %7.4f  I max: %4d  I term: %6.1f",
429             fix2float15(Ki), i_max, fix2float15(i_term)) ;
430         writeString(screentext) ;
431         // ===== END ADDED =====
432         // ===== END ADDED =====
433     }
434
435     PT_END(pt) ;
436 }
437
438 // User input thread. User can change draw speed
439 static PT_THREAD (protothread_serial(struct pt *pt))
440 {
441     PT_BEGIN(pt) ;
442     static char classifier ;
443     static int test_in ;
444     static float float_in ;
445     while(1) {
446         sprintf(pt_serial_out_buffer, "input a command: ");
447         serial_write ;
448         // spawn a thread to do the non-blocking serial read
449         serial_read ;
450         // convert input string to number
451         sscanf(pt_serial_in_buffer,"%c", &classifier) ;
452
453         // num_independents = test_in ;
454         if (classifier=='t') {
455             sprintf(pt_serial_out_buffer, "timestep: ");
456             serial_write ;
457             serial_read ;
458             // convert input string to number
459             sscanf(pt_serial_in_buffer,"%d", &test_in) ;
460             if (test_in > 0) {
461                 threshold = test_in ;
462             }
463         }
464         // ===== ADDED: 'n' command sets fan duty cycle (from PWM_Demo) =====
465         else if (classifier=='n') {
466             // CHANGED: limit is DUTY_MAX (was WRAPVAL)
467             sprintf(pt_serial_out_buffer, "duty cycle (0-%d): ", DUTY_MAX);
```

```
            serial_write ;
            serial_read ;
            // convert input string to number
            sscanf(pt_serial_in_buffer,"%d", &test_in) ;
            if (test_in >= 0 && test_in <= DUTY_MAX) {
                closed_loop = 0 ;   // ADDED: manual duty turns off the controller
                control = test_in ;
            }
        }
        // ===== END ADDED =====
        // ===== ADDED: controller commands =====
        // 'a' = target angle, 'k' = Kp, 'd' = Kd, 'i' = Ki, 'm' = I term cap, 'c' = closed loop on (1)
 / off (0)
        else if (classifier=='a') {
            sprintf(pt_serial_out_buffer, "target angle (-90 to 90 deg): ");
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%f", &float_in) ;
            if (float_in >= -90.0f && float_in <= 90.0f) {
                desired_angle = float2fix15(float_in) ;
            }
        }
        else if (classifier=='p') {
            sprintf(pt_serial_out_buffer, "Kp (duty counts per degree): ");
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%f", &float_in) ;
            if (float_in >= 0.0f && float_in <= 6000.0f) {
                Kp = float2fix15(float_in) ;
            }
        }
        // ===== ADDED: 'd' command sets Kd =====
        else if (classifier=='d') {
            sprintf(pt_serial_out_buffer, "Kd (duty counts per deg/s): ");
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%f", &float_in) ;
            if (float_in >= 0.0f && float_in <= 6000.0f) {
                Kd = float2fix15(float_in) ;
            }
        }
        // ===== END ADDED =====
        // ===== ADDED: 'i' sets Ki, 'm' sets the I term cap =====
        else if (classifier=='i') {
            sprintf(pt_serial_out_buffer, "Ki (duty counts per deg per ms, e.g. 0.03): ");
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%f", &float_in) ;
            if (float_in >= 0.0f && float_in <= 100.0f) {
                Ki = float2fix15(float_in) ;
                if(float_in == 0.0f) { // reset the integral term if Ki is set to 0
                    i_term = 0 ;
                }
            }
        }
        else if (classifier=='m') {
            sprintf(pt_serial_out_buffer, "I term max (0-%d): ", DUTY_MAX);
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%d", &test_in) ;
            if (test_in >= 0 && test_in <= DUTY_MAX) {
                i_max = test_in ;
            }
        }
        // ===== END ADDED =====
        else if (classifier=='c') {
            sprintf(pt_serial_out_buffer, "closed loop (1 = on, 0 = off): ");
            serial_write ;
            serial_read ;
            sscanf(pt_serial_in_buffer,"%d", &test_in) ;
```

```
            if (test_in == 0) {
                closed_loop = 0 ;
                control = 0 ;   // fan off when leaving closed loop
            }
            else if (test_in == 1) {
                closed_loop = 1 ;
            }
        }
        // ===== END ADDED =====
    }
    PT_END(pt) ;
}

// Entry point for core 1
void core1_entry() {
    // REMOVED: GPIO 15 debug output init, GPIO 15 is now the button
    pt_add_thread(protothread_draw) ;
    pt_schedule_start ;
}

int main() {

    // Overclock
    set_sys_clock_khz(150000, true) ;

    // Initialize stdio
    stdio_init_all();

    // Initialize VGA
    initVGA() ;

    // ===== ADDED: I2C bus recovery =====
    // If the Pico resets mid-transfer, the MPU6050 (which stays powered) can be
    // left holding SDA low, and mpu6050_reset() then blocks forever. Clocking SCL
    // up to 9 times lets the IMU finish its byte and release SDA.
    gpio_init(SDA_PIN) ;
    gpio_init(SCL_PIN) ;
    gpio_set_dir(SDA_PIN, GPIO_IN) ;
    gpio_pull_up(SDA_PIN) ;
    gpio_set_dir(SCL_PIN, GPIO_OUT) ;
    gpio_put(SCL_PIN, 1) ;
    sleep_us(10) ;
    for (int k = 0; k < 9 && !gpio_get(SDA_PIN); k++) {
        gpio_put(SCL_PIN, 0) ;
        sleep_us(5) ;
        gpio_put(SCL_PIN, 1) ;
        sleep_us(5) ;
    }
    printf("\n\rI2C bus %s\n\r", gpio_get(SDA_PIN) ? "free" : "STUCK (SDA held low)") ;
    // ===== END ADDED =====

    ////////////////////////////////////////////////////////////////////////
    ////////////////////////// I2C CONFIGURATION ///////////////////////////
    i2c_init(I2C_CHAN, I2C_BAUD_RATE) ;
    gpio_set_function(SDA_PIN, GPIO_FUNC_I2C) ;
    gpio_set_function(SCL_PIN, GPIO_FUNC_I2C) ;

    // MPU6050 initialization
    mpu6050_reset();
    mpu6050_read_raw(acceleration, gyro);

    // // ===== ADDED: seed filters with first reading so the angle doesn't start at 0 =====
    filtered_ax = acceleration[0] ;
    filtered_az = acceleration[2] ;
    complementary_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_az)), oneeightyoverpi);
    // // ===== END ADDED =====

    ////////////////////////////////////////////////////////////////////////
    ////////////////////////// PWM CONFIGURATION ///////////////////////////
    ////////////////////////////////////////////////////////////////////////
```

```
607     // Button Pin Initialization (before the ISR starts reading it)
608     gpio_init(BUTTON_PIN) ;
609     gpio_set_dir(BUTTON_PIN, GPIO_IN) ;
610     gpio_pull_up(BUTTON_PIN) ;
611
612     // Tell GPIO's 4,5 that they allocated to the PWM
613     gpio_set_function(5, GPIO_FUNC_PWM);
614     gpio_set_function(PWM_OUT, GPIO_FUNC_PWM);  // CHANGED: was hardcoded 4, now uses PWM_OUT
615
616     // Find out which PWM slice is connected to GPIO 5 (it's slice 2, same for 4)
617     slice_num = pwm_gpio_to_slice_num(5);
618
619     // Mask our slice's IRQ output into the PWM block's single interrupt line,
620     // and register our interrupt handler
621     pwm_clear_irq(slice_num);
622     pwm_set_irq_enabled(slice_num, true);
623     irq_set_exclusive_handler(PWM_IRQ_WRAP, on_pwm_wrap);
624     irq_set_enabled(PWM_IRQ_WRAP, true);
625
626     // This section configures the period of the PWM signals
627     pwm_set_wrap(slice_num, WRAPVAL) ;
628     pwm_set_clkdiv(slice_num, CLKDIV) ;
629
630     // This sets duty cycle
631     pwm_set_chan_level(slice_num, PWM_CHAN_B, 0);
632     pwm_set_chan_level(slice_num, PWM_CHAN_A, 0);
633
634     // Start the channel
635     pwm_set_mask_enabled((1u << slice_num));
636
637
638     ////////////////////////////////////////////////////////////////////////
639     ////////////////////////////// ROCK AND ROLL ///////////////////////////
640     ////////////////////////////////////////////////////////////////////////
641     // start core 1
642     multicore_reset_core1();
643     multicore_launch_core1(core1_entry);
644
645     // start core 0
646     gpio_init(14) ;
647     gpio_set_dir(14, GPIO_OUT) ;
648     gpio_put(14, 0) ;
649
650     pt_add_thread(protothread_serial) ;
651     pt_schedule_start ;
652
653 }
```

A.2 mpu6050.c (IMU driver, course code)

```
1  /**
2   * Hunter Adams (vha3@cornell.edu)
3   *
4   *
5   */
6
7  #include "hardware/i2c.h"
8  #include "mpu6050.h"
9
10 void mpu6050_reset() {
11     // Two byte reset. First byte register, second byte data
12     // There are a load more options to set up the device in
13     // different ways that could be added here
14     uint8_t buf[] = {0x6B, 0x00};
15     i2c_write_blocking(I2C_CHAN, ADDRESS, buf, 2, false);
16
17     // Set gyro sample rate (set to 1KHz, same as accel)
18     uint8_t gyro_rate[] = {0x19, 0b00000111} ;
19     i2c_write_blocking(I2C_CHAN, ADDRESS, gyro_rate, 2, false);
```

```

    // Configure the Gyro range (+/- 250 deg/s)
    uint8_t gyro_settings[] = {0x1b, 0b00000000} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, gyro_settings, 2, false);

    // Configure the Accel range (+/- 2g's)
    uint8_t accel_settings[] = {0x1c, 0b00000000} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, accel_settings, 2, false);

    // Configure interrupt pin
    uint8_t pin_settings[] = {0x37, 0b00010000} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, pin_settings, 2, false);

    // Configure data ready interrupt
    uint8_t int_config[] = {0x38, 0x01} ;
    i2c_write_blocking(I2C_CHAN, ADDRESS, int_config, 2, false);
}

void mpu6050_read_raw(fix15 accel[3], fix15 gyro[3]) {
    // For this particular device, we send the device the register we want to read
    // first, then subsequently read from the device. The register is auto incrementing
    // so we don't need to keep sending the register we want, just the first.

    uint8_t buffer[6];
    int16_t temp_accel, temp_gyro ;

    // Status value
    uint32_t success;

    // Start reading acceleration registers from register 0x3B for 6 bytes
    uint8_t val = 0x3B;
    i2c_write_timeout_us(I2C_CHAN, ADDRESS, &val, 1, true, 250); // true to keep master control of bus
    i2c_read_timeout_us(I2C_CHAN, ADDRESS, buffer, 6, false, 250);

    for (int i = 0; i < 3; i++) {
        temp_accel = (buffer[i<<1] << 8 | buffer[(i<<1) + 1]);
        accel[i] = temp_accel ;
        accel[i] <<= 2 ; // convert to g's (fixed point)
    }

    // Now gyro data from reg 0x43 for 6 bytes
    // The register is auto incrementing on each read
    val = 0x43;
    i2c_write_timeout_us(I2C_CHAN, ADDRESS, &val, 1, true, 250);
    i2c_read_timeout_us(I2C_CHAN, ADDRESS, buffer, 6, false, 250);  // False - finished with bus

    for (int i = 0; i < 3; i++) {
        temp_gyro = (buffer[i<<1] << 8 | buffer[(i<<1) + 1]);
        gyro[i] = temp_gyro ;
        gyro[i] = multfix15(gyro[i], 500<<16) ; // deg/sec
    }
}
////////////////////////////////////////////////////////////////
```

A.3 mpu6050.h (pins, fix15 macros and constants)

```
/**
 * Hunter Adams (vha3@cornell.edu)
 *
 *
 */

#define ADDRESS 0x68
#define I2C_CHAN i2c0
#define SDA_PIN  8
#define SCL_PIN  9
#define I2C_BAUD_RATE 400000

// Fixed point data type
```

```
typedef signed int fix15 ;
#define multfix15(a,b) ((fix15)(((( signed long long)(a))*(( signed long long)(b)))>>16))
#define float2fix15(a) ((fix15)((a)*65536.0f)) // 2^16
#define fix2float15(a) ((float)(a)/65536.0f)
#define int2fix15(a) ((a)<<16)
#define fix2int15(a) ((a)>>16)
#define divfix(a,b) ((fix15)(((( signed long long)(a) << 16 / (b)))))
// Parameter values
#define oneeightyoverpi 3754936
#define zeropt001 65
#define zeropt999 65470
#define zeropt01 655
#define zeropt99 64880
#define zeropt1 6553
#define zeropt9 58982

// VGA primitives - usable in main
void mpu6050_reset(void) ;
void mpu6050_read_raw(fix15 accel[3], fix15 gyro[3]) ;
```

A.4 CMakeLists.txt (build)

```
# == DO NOT EDIT THE FOLLOWING LINES for the Raspberry Pi Pico VS Code Extension to work ==
if(WIN32)
    set(USERHOME $ENV{USERPROFILE})
else()
    set(USERHOME $ENV{HOME})
endif()
set(sdkVersion 2.3.1)
set(toolchainVersion 15_2_Rel1)
set(picotoolVersion 2.3.1)
set(picoVscode ${USERHOME}/.pico-sdk/cmake/pico-vscode.cmake)
if (EXISTS ${picoVscode})
    include(${picoVscode})
endif()
# ====================================================================================
set(PICO_BOARD pico2 CACHE STRING "Board type")

# cmake version
cmake_minimum_required(VERSION 3.13)

# include the sdk.cmake file
include(pico_sdk_import.cmake)

# give the project a name (anything you want)
project(MPU6050_IMU C CXX ASM)

# initialize the sdk
pico_sdk_init()

add_executable(MPU6050_IMU)

# must match with pio filename and executable name from above
pico_generate_pio_header(MPU6050_IMU ${CMAKE_CURRENT_LIST_DIR}/VGA/hsync.pio)
pico_generate_pio_header(MPU6050_IMU ${CMAKE_CURRENT_LIST_DIR}/VGA/vsync.pio)
pico_generate_pio_header(MPU6050_IMU ${CMAKE_CURRENT_LIST_DIR}/VGA/rgb.pio)

# must match with executable name and source file names
target_sources(MPU6050_IMU PRIVATE imu_demo.c VGA/vga16_graphics_v3.c mpu6050.c)

# Add pico_multicore which is required for multicore functionality
target_link_libraries(  MPU6050_IMU
                        pico_stdlib
                        pico_bootsel_via_double_reset
                        pico_multicore
                        hardware_pwm
                        hardware_dma
                        hardware_irq
                        hardware_adc
```

```
                        hardware_pio
                        hardware_sync
                        hardware_i2c
                        hardware_clocks)

# create map/bin/hex file etc.
pico_add_extra_outputs(MPU6050_IMU)
```

Appendix B AI Prompt Log

Tool: Claude Code in VS Code, model **Claude Opus 5.5**. **Dates:** October 21–22, 2026. **Source:** the prompt log Claude kept in our project (`MPU6050_IMU_Demo/PROMPT_LOG.md`), copied below. Prompts are word for word, typos included. Claude kept the log as it went (from prompt 9 on, filling in 1–8 afterwards), so some responses are shorter summaries of the full chat reply.

How we counted. *Words* are counted from the log (our prompt, and Claude's response without code). *Code lines given* counts the C lines in each response. One *suggested* addition, deletion or change is one distinct change to the program (like a new serial command or one edited line), not one line of text. *Accepted* means it's still in the final `imu_demo.c` (Appendix A), even if we edited it later. Things we used and then removed count as not accepted.

#	Date	Prompt (topic)	Prompt words	Response words	Code lines given	Suggested add / del / mod	Accepted add / del / mod	Notes
1	10/21	Integrate PWM fan control into the IMU demo	11	57	23	3 / 0 / 1	3 / 0 / 1	Kept. We later renamed the manual-duty command from `p` to `n`.
2	10/21	Comment which code was added	11	32	0	0 / 0 / 0	0 / 0 / 0	Comment markers only; kept (the ADDED / CHANGED markers in Appendix A).
3	10/21	Fan runs at max with duty cycle 0	23	69	0	0 / 0 / 0	0 / 0 / 0	Diagnosis only.
4	10/21	6000 also runs at full speed; PWM_Demo works	35	92	0	0 / 0 / 0	0 / 0 / 0	Diagnosis only (program hung in I2C setup).
5	10/21	Add a complementary filter and display it	14	80	38	9 / 0 / 0	7 / 0 / 0	`#define PI` and the magenta accel-angle trace were later removed.
6	10/21	VGA isn't displaying anything	10	100	5	0 / 1 / 1	0 / 1 / 1	Kept: local Q15 macros deleted, 180/π constant fixed.
7	10/21	Serial terminal broken too	10	60	29	4 / 0 / 2	0 / 0 / 0	WHO_AM_I check and `imu_ok` guards later removed by us; replaced by entry 10.
8	10/21	Plots all read 0	9	77	2	0 / 0 / 1	0 / 0 / 0	Removed together with the entry 7 check.
9	10/21	Create this prompt log	37	21	0	0 / 0 / 0	0 / 0 / 0	Created PROMPT_LOG.md.
10	10/21	VGA not displaying again (IMU wiring checked)	26	74	14	1 / 0 / 0	1 / 0 / 0	Kept: I2C bus recovery at start-up.
11	10/21	How does the complementary filter work / rotation about +y	30	192	7	1 / 0 / 5	1 / 0 / 5	Offered as a suggestion; we applied the axis change by hand (finished in entry 12).
12	10/21	Angles read 80–300° and don't track (arm rotates about +y)	75	105	12	0 / 0 / 4	0 / 0 / 4	Kept: a_z axis, +π removed, plot offset removed, seed re-enabled.
13	10/21	Closed-loop proportional control of the motor	13	147	34	7 / 0 / 3	7 / 0 / 3	Kept. We later renamed the Kp command from `k` to `p`.
14	10/22	How to tune Kp (bouncing vs. settling ~10° low)	39	134	0	1 / 0 / 0	0 / 0 / 0	PID + feed-forward sketch not used.
15	10/22	Add derivative control	23	104	18	3 / 0 / 2	3 / 0 / 2	Kept.
16	10/22	Clamp the duty cycle to 4500	9	39	8	1 / 0 / 2	1 / 0 / 2	Kept, but we later raised `DUTY_MAX` back to 6000.
17	10/22	Add integral control	6	147	24	6 / 0 / 1	6 / 0 / 1	Kept. We later raised `i_max` to 10000 and made `i 0` clear the integrator.

#	Date	Prompt (topic)	Prompt words	Response words	Code lines given	Suggested add / del / mod	Accepted add / del / mod	Notes
18	10/22	Tuning PID for fast response	20	189	0	0 / 0 / 0	0 / 0 / 0	Tuning advice only.
Total (18 prompts)			**401**	**1719**	**214**	**36 / 1 / 22**	**29 / 1 / 19**	

Table B1. Prompt log summary (18 prompts). Totals: suggested 36 / 1 / 22, accepted 29 / 1 / 19 (additions / deletions / changes).

Note: the demo-button sequence and the overcurrent protection (ramp limit and the windup fix) are also marked `ADDED` in Appendix A, but they were written after the last entry in `PROMPT_LOG.md`, so they aren't in the log or counted in Table B1.

Full log

B.1 Integrate PWM fan control into the IMU demo

2026-10-21 · 11 prompt words · 57 response words

OUR PROMPT (VERBATIM)

> *Can you integrate the pwm fan control into the imu demo*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

The IMU demo already set up the PWM slice on GPIO 4/5 (and used the PWM wrap interrupt to read the IMU at 1 kHz), but the duty cycle was stuck at 0. Merged the duty-cycle update from `PWM_Demo/pwm_demo.c` into the existing ISR and added a serial command `p` to set the duty cycle (0–6000). Built successfully.

CODE (IMU_DEMO.C):

```
// Globals
volatile int control ;
volatile int old_control ;

// In on_pwm_wrap()
if (control!=old_control) {
    old_control = control ;
    pwm_set_chan_level(slice_num, PWM_CHAN_A, control);
}

// In protothread_serial
else if (classifier=='p') {
    sprintf(pt_serial_out_buffer, "duty cycle (0-%d): ", WRAPVAL);
    serial_write ;
    serial_read ;
    sscanf(pt_serial_in_buffer,"%d", &test_in) ;
    if (test_in >= 0 && test_in <= WRAPVAL) {
        control = test_in ;
    }
}

// In main()
gpio_set_function(PWM_OUT, GPIO_FUNC_PWM);  // was hardcoded 4
```

B.2 Comment which code was added

2026-10-21 · 11 prompt words · 32 response words

OUR PROMPT (VERBATIM)

> *Can you clearly comment which parts of the code you added*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Wrapped each added block in `// ===== ADDED: ... =====` / `// ===== END ADDED =====` markers, and marked the one changed line with `// CHANGED: was hardcoded 4, now uses PWM_OUT`.

CODE: COMMENT MARKERS ONLY.

B.3 Fan runs at max with duty cycle 0

2026-10-21 · 23 prompt words · 69 response words

OUR PROMPT (VERBATIM)

> *something is wrong with the pwm control. when inputtitng a duty cycle of 0 it is still running the fan at max value*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Traced the code path. It correctly sets GPIO 4's level to 0, and the startup level is also 0, so the software logic looked right. Suggested hardware checks: does the fan run at full speed before any input, is the driver circuit inverting (fix: `pwm_set_output_polarity(slice_num, true, false)`), does PWM_Demo behave the same, is the fan on GPIO 4, is the MOSFET gate floating, and was the new `.uf2` flashed.

CODE: NONE.

B.4 6000 also runs at full speed; PWM_Demo works

2026-10-21 · 35 prompt words · 92 response words

OUR PROMPT (VERBATIM)

> *6000 still seems to run it at full speed. running the PWM_Demo itself works and lets you change hte fan speed. could it be somehting changing the logic hwen merging the pwm code into imu?*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Compared the merged code against PWM_Demo. The duty-cycle logic, pin, wrap and clock divider were identical. There were no pin conflicts (I2C on 8/9, VGA on 16–21, UART on 0/1), and both builds targeted the Pico 2 (RP2350). Concluded the IMU program probably never reached the PWM setup. `main()` sets up VGA and the IMU first, and `mpu6050_reset()` uses I2C writes with no timeout, which hang if the bus is stuck, so GPIO 4 would float. Asked whether the serial prompt and the VGA traces appear and whether the IMU was connected.

CODE: NONE.

B.5 Add a complementary filter and display it

2026-10-21 · 14 prompt words · 80 response words

OUR PROMPT (VERBATIM)

> *how do I properly add a complementary filter and see it on the screen*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

The user had started adding the filter lines, but the variables were undeclared and the low-pass filter was missing. Added the declarations, a first-order IIR low-pass filter (alpha = 1/16) on the accelerometer x/y, starting values seeded from the first reading, display buffers, magenta (accel-only) and yellow (complementary) traces on the top plot shifted by −180°, and a numeric text readout. Noted that the gyro sign, the axis choice, the 1 kHz timestep and the 0.999/0.001 blend may need tuning.

CODE (IMU_DEMO.C):

```
#define PI 3.14159265
fix15 filtered_ax, filtered_ay ;
fix15 accel_angle ;
fix15 gyro_angle_delta ;
fix15 complementary_angle ;
fix15 accel_angle_buf[PLOT_WIDTH] ;
fix15 comp_angle_buf[PLOT_WIDTH] ;

// ISR: low pass filter
filtered_ax = filtered_ax + ((acceleration[0] - filtered_ax) >> 4) ;
filtered_ay = filtered_ay + ((acceleration[1] - filtered_ay) >> 4) ;

// ISR: store for display
accel_angle_buf[head] = accel_angle ;
comp_angle_buf[head] = complementary_angle ;
```

```
// Draw thread: angle traces
drawPixel(LEFT_EDGE + i, (TOP_PLOT_BOTTOM -
    (int)(NEW_RANGE*((float)((fix2float15(accel_angle_buf[start]) - 180.)-OLD_MIN)/OLD_RANGE))), MAGENTA) ;
drawPixel(LEFT_EDGE + i, (TOP_PLOT_BOTTOM -
    (int)(NEW_RANGE*((float)((fix2float15(comp_angle_buf[start]) - 180.)-OLD_MIN)/OLD_RANGE))), YELLOW) ;

// Draw thread: text readout
setTextSize(2) ;
setTextColor(MAGENTA) ;
setCursor(85, 20) ;
sprintf(screentext, "Accel: %6.1f", fix2float15(accel_angle)) ;
writeString(screentext) ;
setTextColor(YELLOW) ;
setCursor(330, 20) ;
sprintf(screentext, "Comp: %6.1f", fix2float15(complementary_angle)) ;
writeString(screentext) ;
setTextSize(1) ;

// main(): seed filters
filtered_ax = acceleration[0] ;
filtered_ay = acceleration[1] ;
complementary_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_ay) + PI), oneeightyoverpi);
```

B.6 VGA isn't displaying anything

2026-10-21 · 10 prompt words · 100 response words

OUR PROMPT (VERBATIM)

> *can you check for bugs the vga isn't displaying anything*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Found two math bugs:

1. A local set of Q15 fixed-point macros (1.0 = 32768) was overriding the Q16 macros in `mpu6050.h` (1.0 = 65536), while `mpu6050.c` still produced Q16 data, so every value was misread by 2×. Removed the local macros.
2. `multfix15(..., 180 / M_PI)` truncated the constant to the integer 57. Replaced it with `oneeightyoverpi`.

Neither would blank the screen: `drawPixel` checks its bounds and the VGA driver is DMA-only. Concluded the program was likely hung in `mpu6050_reset()` before core 1 started, and suggested checking for the serial prompt, scoping GPIO 14/15, and checking the IMU wiring.

CODE (IMU_DEMO.C):

```
// FIXED: removed local Q15 fixed-point macros. mpu6050.c / mpu6050.h use
// 15.16 format (1.0 = 65536), so the IMU data must be read with those macros.

// FIXED: second argument must be fixed point (180/M_PI was truncated to the integer 57)
accel_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_ay) + PI), oneeightyoverpi);
```

B.7 Serial terminal broken too

2026-10-21 · 10 prompt words · 60 response words

OUR PROMPT (VERBATIM)

> *the serial command terminal seems to be broken as well*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

The serial terminal, VGA and fan all failing together confirmed the program was hanging during startup in the I2C/IMU setup. Added a WHO_AM_I check with a timeout, an `imu_ok` flag that skips the IMU reset and the ISR reads if the IMU doesn't respond (so the timeouts can't starve core 0), and startup progress messages printed to the serial terminal.

CODE (IMU_DEMO.C):

```
// Global
volatile int imu_ok = 0 ;
```

```
// main(), before mpu6050_reset()
printf("\n\rVGA and I2C initialized\n\r") ;
uint8_t who_reg = 0x75 ;
uint8_t who_am_i = 0 ;
int w = i2c_write_timeout_us(I2C_CHAN, ADDRESS, &who_reg, 1, true, 1000) ;
int r = i2c_read_timeout_us(I2C_CHAN, ADDRESS, &who_am_i, 1, false, 1000) ;
if (w == 1 && r == 1 && who_am_i == 0x68) {
    imu_ok = 1 ;
    printf("MPU6050 found (WHO_AM_I = 0x%02x)\n\r", who_am_i) ;
}
else {
    imu_ok = 0 ;
    printf("ERROR: MPU6050 not responding (write=%d read=%d WHO_AM_I=0x%02x)\n\r", w, r, who_am_i) ;
    printf("Check IMU power/ground, SDA on GPIO %d, SCL on GPIO %d\n\r", SDA_PIN, SCL_PIN) ;
}
if (imu_ok) {
    mpu6050_reset();
    mpu6050_read_raw(acceleration, gyro);
    printf("MPU6050 reset done\n\r") ;
}

// ISR
if (imu_ok) mpu6050_read_raw(acceleration, gyro);

// After multicore_launch_core1()
printf("Core 1 launched, starting serial thread\n\r") ;
```

B.8 Plots all read 0

2026-10-21 · 9 prompt words · 77 response words

OUR PROMPT (VERBATIM)

> *The plots are all just reading at 0 now*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

The WHO_AM_I check was too strict. Clone boards often report an ID other than `0x68` (such as `0x70`, `0x72` or `0x98`), so `imu_ok` stayed 0 and every IMU read was skipped. Relaxed the check to accept any device that acknowledges on I2C. Explained how to read the error codes in the ERROR message: `-1` means the IMU didn't respond at its address, `-2` means the bus is stuck (timeout). Also suggested checking the AD0 pin (address 0x69).

CODE (IMU_DEMO.C):

```
// Accept any device that answers: clone boards often report an ID other than 0x68
if (w == 1 && r == 1) {
```

B.9 Create this prompt log

2026-10-21 · 37 prompt words · 21 response words

OUR PROMPT (VERBATIM)

> *Can you record all prompts in a markdown file for the previous prompts and for future prompts. And keep a prompt log of the promt and the answer and code if generated as well as the date.*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Created this file (`PROMPT_LOG.md`) with every prompt so far, and saved a memory note so future sessions keep appending entries here.

CODE: NONE (THIS FILE).

B.10 VGA not displaying again (IMU wiring checked)

2026-10-21 · 26 prompt words · 74 response words

OUR PROMPT (VERBATIM)

> *The VGA screen is not displaying again. previously this was because the imu wiring became loose but now I checked this and its still not working.*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

The user had removed the WHO_AM_I check and the `imu_ok` guards, so `mpu6050_reset()` could block forever again. The likely cause was a stuck I2C bus: when the Pico resets mid-transfer, the still-powered MPU6050 can hold SDA low, and reflashing doesn't clear it (a power cycle does). Suggested unplugging USB as a quick test, and added a standard I2C bus recovery at startup (up to 9 SCL pulses) that prints whether the bus is free.

CODE (IMU_DEMO.C, IN MAIN() BEFORE I2C_INIT):

```
gpio_init(SDA_PIN) ;
gpio_init(SCL_PIN) ;
gpio_set_dir(SDA_PIN, GPIO_IN) ;
gpio_pull_up(SDA_PIN) ;
gpio_set_dir(SCL_PIN, GPIO_OUT) ;
gpio_put(SCL_PIN, 1) ;
sleep_us(10) ;
for (int k = 0; k < 9 && !gpio_get(SDA_PIN); k++) {
    gpio_put(SCL_PIN, 0) ;
    sleep_us(5) ;
    gpio_put(SCL_PIN, 1) ;
    sleep_us(5) ;
}
printf("\n\rI2C bus %s\n\r", gpio_get(SDA_PIN) ? "free" : "STUCK (SDA held low)") ;
```

B.11 How does the complementary filter work / rotation about +y

2026-10-21 · 30 prompt words · 192 response words

OUR PROMPT (VERBATIM)

> *Can you explain how the complementary filter works? What should we see the accel angle and complementary angle change as we rotate about the + y axis of the imu*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

- **Concept:** the accel angle (from gravity) never drifts but is noisy, and the gyro integral is smooth but drifts. The filter `angle = 0.999*(angle + rate*dt) + 0.001*accel_angle` acts as a high-pass on the gyro and a low-pass on the accel, with about a 1 s crossover (dt/0.001).
- **Current code:** it measures rotation about **z** (accel x/y, gyro z). Rotating about +y moves gravity between x and z instead, so with a flat IMU the accel angle flips between about 90° and 270°, and the complementary angle gets no gyro input and just drifts slowly toward it. The start-up seed is also commented out, so the complementary angle starts at 0 and takes about 3–5 s to settle.
- **For +y rotation:** use accel x/z and gyro y (code below, suggested only, not applied). Expected behaviour: flat = 0°, a +30° tilt reads +30° on both traces, accel is jittery while the complementary trace is smooth, a fast tilt is tracked with no lag, vibration spikes only the accel, and there's no long-term drift. If the complementary trace moves the wrong way and then creeps back, flip the gyro sign.

CODE (SUGGESTED, NOT APPLIED):

```
filtered_ax = filtered_ax + ((acceleration[0] - filtered_ax) >> 4) ;
filtered_az = filtered_az + ((acceleration[2] - filtered_az) >> 4) ;
accel_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_az)), oneeightyoverpi);
gyro_angle_delta = multfix15(gyro[1], zeropt001) ;
complementary_angle = multfix15(complementary_angle + gyro_angle_delta, zeropt999)
                    + multfix15(accel_angle, zeropt001);
// also: declare fix15 filtered_az; remove "- 180." from the angle plot lines
```

B.12 Angles read 80–300° and don't track (arm rotates about +y)

2026-10-21 · 75 prompt words · 105 response words

OUR PROMPT (VERBATIM)

> *The IMU setup is an arm that rotates about the +y axis from -180 to 180 deg in the z axis, so the IMU chip lays flat when the arm is held horizontally in the middle of its rotation range. Right now accel ang and comp ang dont seem to*

> *make sense because htey range from 80 to 300 deg and dont really track the intermediate movement. is the orientation in the code still wrong?*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Yes. The user had renamed the variable to `filtered_az` and switched to gyro y with a `+`, but `filtered_az` was still filtering `acceleration[1]` (the y axis, which is the rotation axis and stays near 0). With a near-zero denominator, `atan2` flipped toward ±90° (90°/270° after the +PI offset), which matched the 80–300° symptom. Fixed it to use `acceleration[2]`, removed the `+PI` so flat = 0° and the range is ±180°, removed the `-180.` plot shift, and re-enabled the start-up seed using the z axis. It builds cleanly. Noted the wraparound at ±180° (arm pointing straight down), where the accel angle jumps between +180 and −180.

CODE (IMU_DEMO.C):

```
// ISR
filtered_az = filtered_az + ((acceleration[2] - filtered_az) >> 4) ;
accel_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_az)), oneeightyoverpi);

// Draw thread (no -180 shift)
(int)(NEW_RANGE*((float)((fix2float15(accel_angle_buf[start]))-OLD_MIN)/OLD_RANGE))
(int)(NEW_RANGE*((float)((fix2float15(comp_angle_buf[start]))-OLD_MIN)/OLD_RANGE))

// main() seed
filtered_ax = acceleration[0] ;
filtered_az = acceleration[2] ;
complementary_angle = multfix15(float2fix15(atan2(-filtered_ax, filtered_az)), oneeightyoverpi);
```

B.13 Closed-loop proportional control of the motor

2026-10-21 · 13 prompt words · 147 response words

OUR PROMPT (VERBATIM)

> *How do we add closed loop proportional control of the motor using PWM?*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Added a P controller in the 1 kHz IMU ISR, right after the complementary angle update: `error = desired - complementary_angle`, `duty = Kp * error`, clamped to 0..6000 (the fan only pushes one way). Moved the PWM level update after the controller so each new value is applied in the same interrupt. Added serial commands: `a` target angle, `k` Kp, `c` closed loop on/off (turning it off sets the duty to 0), and `p` manual duty, which now turns the closed loop off. Added a light-blue target line on the top plot and a readout of target, Kp, duty and mode. Enlarged `screentext` from 40 to 80 chars, because the readout line would have overflowed it. Explained that a P-only controller leaves a steady-state error because it needs a nonzero error to hold the arm up against gravity, which is what the I term will fix.

CODE (IMU_DEMO.C):

```
// Globals
volatile int closed_loop = 0 ;
volatile fix15 desired_angle = 0 ;
volatile fix15 Kp = int2fix15(100) ;
fix15 angle_error ;
char screentext[80];

// ISR, after complementary filter
if (closed_loop) {
    angle_error = desired_angle - complementary_angle ;
    int u = (int)((((long long)Kp) * ((long long)angle_error)) >> 32) ;
    if (u < 0) u = 0 ;
    if (u > WRAPVAL) u = WRAPVAL ;
    control = u ;
}
if (control!=old_control) {
    old_control = control ;
    pwm_set_chan_level(slice_num, PWM_CHAN_A, control);
}
```

```
// Draw thread
drawHLine(LEFT_EDGE, (TOP_PLOT_BOTTOM -
    (int)(NEW_RANGE*((float)((fix2float15(desired_angle))-OLD_MIN)/OLD_RANGE))), PLOT_WIDTH, LIGHT_BLUE) ;
setTextColor(LIGHT_BLUE) ;
setCursor(85, 45) ;
sprintf(screentext, "Target: %6.1f  Kp: %6.1f  Duty: %4d  %s",
    fix2float15(desired_angle), fix2float15(Kp), control, closed_loop ? "CLOSED LOOP" : "MANUAL") ;
writeString(screentext) ;

// Serial thread
else if (classifier=='a') { /* read float -> desired_angle (-180..180) */ }
else if (classifier=='k') { /* read float -> Kp (0..6000) */ }
else if (classifier=='c') { /* 1 = closed_loop on, 0 = off and control = 0 */ }
// 'p' now also sets closed_loop = 0
```

B.14 How to tune Kp (bouncing vs. settling ~10° low)

2026-10-22 · 39 prompt words · 134 response words

OUR PROMPT (VERBATIM)

> *We added proportional closed loop control. how do we tune kp for the fan to drive the arm to a specified angle? Sometimes it bounces and sometimes its not strong enough to reach the desired angle (~10 deg lower).*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Both symptoms are limits of P-only control, so no single Kp fixes both. The arm needs a nonzero holding duty against gravity, so P settles where `error_ss = u_hold / Kp` (can be checked by finding the holding duty with `p`). Raising Kp shrinks that error, but the arm is a lightly damped pendulum with fan lag, so it overshoots and oscillates, and the `u < 0` clamp lets it free-fall on overshoot. Tuning procedure: raise Kp to sustained oscillation and use about half; add D from the gyro rate to damp it (lets Kp go higher); then add I with anti-windup, or a feedforward holding duty `u_ff`, to remove the offset. Watch the duty readout: swinging 0↔6000 means the gain is too high, pinned at 6000 means the fan can't reach the angle.

CODE (SUGGESTED, NOT APPLIED): PID + FEEDFORWARD SKETCH FOR THE ISR (KD, KI, `U_FF`, CLAMPED `ERROR_INTEGRAL`, `D = KD * GYRO[1]` SUBTRACTED).

B.15 Add derivative control

2026-10-22 · 23 prompt words · 104 response words

OUR PROMPT (VERBATIM)

> *can you add derivative control? also make sure to comment the sections you are changing and add the change log to the markdowns.*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Added a D term to the ISR controller, using the gyro y rate directly (d(error)/dt = −gyro[1] because the target is constant), which avoids the noise of differencing the angle estimate, as recommended in lecture. The control law is now `u = Kp*error - Kd*gyro[1]`, still clamped to 0..6000. Added a `d` serial command (0–6000) to set Kd, which defaults to 0 so behaviour is unchanged until it is set, and added Kd to the VGA readout. Marked the changes with `ADDED` / `CHANGED` comments, added an implementation change log section to `pid_lecture_breakdown.md`, and logged the previous (tuning) prompt as entry 14. Builds cleanly.

CODE (IMU_DEMO.C):

```
// Globals
volatile fix15 Kd = int2fix15(0) ;              // gain (duty counts per deg/s, 15.16), 0 = off

// ISR controller
angle_error = desired_angle - complementary_angle ;
long long p_term = ((long long)Kp) * ((long long)angle_error) ;
long long d_term = ((long long)Kd) * ((long long)gyro[1]) ;
int u = (int)((p_term - d_term) >> 32) ;   // CHANGED: was Kp * error only
if (u < 0) u = 0 ;
if (u > WRAPVAL) u = WRAPVAL ;
control = u ;
```

```
// Draw thread readout
sprintf(screentext, "Target: %6.1f  Kp: %6.1f  Kd: %6.1f  Duty: %4d  %s",
    fix2float15(desired_angle), fix2float15(Kp), fix2float15(Kd), control, closed_loop ? "CLOSED LOOP" :
"MANUAL") ;

// Serial thread
else if (classifier=='d') { /* read float -> Kd (0..6000) */ }
```

B.16 Clamp the duty cycle to 4500

2026-10-22 · 9 prompt words · 39 response words

OUR PROMPT (VERBATIM)

> *can you clamp the duty cycle to max 4500*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Added `#define DUTY_MAX 4500`. The closed-loop clamp now limits `u` to `DUTY_MAX` instead of `WRAPVAL` (6000), and the manual `p` command only accepts 0–4500. The PWM period (`WRAPVAL` = 6000) is unchanged, so 4500 is 75% duty. Builds cleanly.

CODE (IMU_DEMO.C):

```
#define DUTY_MAX 4500

// ISR controller
if (u > DUTY_MAX) u = DUTY_MAX ;   // CHANGED: was WRAPVAL (6000)

// Serial 'p' command
sprintf(pt_serial_out_buffer, "duty cycle (0–%d): ", DUTY_MAX);
if (test_in >= 0 && test_in <= DUTY_MAX) {
```

B.17 Add integral control

2026-10-22 · 6 prompt words · 147 response words

OUR PROMPT (VERBATIM)

> *Now can you add integral control*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Added an I term to the ISR controller, making it a full PID: `u = Kp*error - Kd*gyro[1] + i_term`, clamped to 0..`DUTY_MAX`. Every 1 ms sample the controller accumulates `Ki*error` (rather than the raw error), so the sum is already in duty counts. That means it can't overflow 15.16 fixed point, and changing Ki doesn't make the output jump. Anti-windup: `i_term` is capped at `i_max` (default 3000; the lecture says to set it a little above the duty that holds the arm at the target) and floored at 0, because the fan can't pull down. The integrator resets to 0 whenever closed loop is off (including after `p` or `c 0`). New serial commands: `i` sets Ki (default 0 = off, suggested start ~0.03) and `m` sets `i_max` (0–`DUTY_MAX`). Added a second VGA readout line showing Ki, I max and the live I term. Builds cleanly.

CODE (IMU_DEMO.C):

```
// Globals
volatile fix15 Ki = int2fix15(0) ;
volatile int i_max = 3000 ;
volatile fix15 i_term = 0 ;

// ISR controller
i_term += multfix15(Ki, angle_error) ;
if (i_term > int2fix15(i_max)) i_term = int2fix15(i_max) ;
if (i_term < 0) i_term = 0 ;
int u = (int)((p_term - d_term) >> 32) + fix2int15(i_term) ;
...
else {
    i_term = 0 ;   // controller off -> clear integrator
}

// Draw thread
setCursor(85, 57) ;
sprintf(screentext, "Ki: %7.4f  I max: %4d  I term: %6.1f",
```

```
    fix2float15(Ki), i_max, fix2float15(i_term)) ;
writeString(screentext) ;

// Serial thread
else if (classifier=='i') { /* read float -> Ki (0..100) */ }
else if (classifier=='m') { /* read int -> i_max (0..DUTY_MAX) */ }
```

B.18 Tuning PID for fast response

2026-10-22 · 20 prompt words · 189 response words

OUR PROMPT (VERBATIM)

> *If we want the arm to be very fast response to angle changes, how should we tune the pid parameters*

CLAUDE'S RESPONSE (CLAUDE OPUS 5.5, CLAUDE CODE)

Speed comes from Kp, and Kd is what lets Kp go high without bouncing. Aim for slight overshoot (about 5–10%), which settles faster than a fully damped response. Procedure, using step tests (e.g. target 0 → 30°): raise Kp until it overshoots, raise Kd until the overshoot is damped, and repeat. Stop when either (a) the duty slams between 0 and `DUTY_MAX` during a step, which means the actuator is saturated and more gain won't help, or (b) the fan chatters at rest, which means Kd is amplifying gyro noise (a low-pass filter on gyro[1] would help). Add Ki last and keep it small. I is slow and winds up during large steps, which causes overshoot, so keep `i_max` tight (holding duty + margin). Physical limits: the rise is limited by `DUTY_MAX` (4500) and the fall by gravity, because the fan can't pull down, so the response is asymmetric. Options for more speed: feedforward of the holding duty, so P doesn't have to do that job; integrating only when the error is small (conditional integration); raising `DUTY_MAX`. Use `t` to slow the plot scroll and see the step response.

CODE: NONE.